

CT-API

설계 및 구현

CONTENTS

SURESOFT

01 CT-API 설계

02 CT-API 구현 및 개선

03 CT-API 결과

01 CT-API 설계

SURESOFT

CT측 요구사항

AI 도우미 - 코드 기반 테스트 코드 생성

구문 커버리지

Generate Code-Based Tests with AI

테스트 생성 방법

☐ 구조 기반

Goal: 구문 커버리지 90 %

☐ 요구사항 문서 기반

☒ V-SPICE 명세 기반

URL:

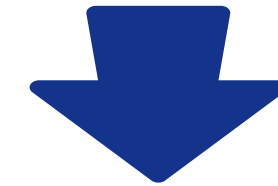
UserName:

Password:

Next > Cancel

무엇을 만들기 위해?

AI 도우미 - 코드 기반 테스트 코드 생성 기능



V-SPICE의 테스트 케이스 명세 기반을 토대로
AI를 활용해 테스트 케이스를 추가로 더 만드는 기능

이를 위해 V-SPICE 측에서 Design의 TestCase를 선택 전달해주는 API 필요

CT측 요구사항

사용자 시나리오 분석

Generate Code-Based Tests with AI

테스트 생성 방법

☒ 구조 기반

Goal: 구문 커버리지 90 %

☐ 요구사항 문서 기반

☒ V-SPICE 명세 기반

URL: _____

UserName: _____

Password: _____

Next > Cancel

- V-SPICE 명세 기반 생성을 선택하여 V-SPICE 정보 입력
 - URL
 - 사용자
 - 암호
- [Next >] 선택 시 Design 목록 요청

1. V-SPICE 로그인 정보 입력

2. 로그인 성공 Design 목록 로드 및 선택 가능

Generate Code-Based Tests with AI

Design: Test

File: All

Function: All

File	Function	Test case	Description
☒ \ap_canComm	void CC_CanBusOffSt	001	CAN 버스 오프 상태 지속 시간 카운
☐ \ap_canComm	void CC_CanBusOffSt	002	CAN 버스 오프 상태 (CanBusOffSt

< Prev Ok Cancel

- Design
 - 기본값: 빈값
 - Select: 선택된 Design 데이터 요청
 - 소스 파일, 함수, TC ID, TC Description (CT UI에 표시되는 정보)
- File/Function
 - Design이 선택되면 기본값으로 자동 설정
 - 기본값: All

3. 해당 Design의 File과 그 File의 함수 선택 가능 (기본값 All/All) ⇒ Test Case 목록 로드

4. 필요한 Test Case들 선택

Generate Code-Based Tests with AI

Design: Test

File: All

Function: All

filter...

File	Function	Test case	Description
☒ \ap_canComm	void CC_CanBusOffSt	001	CAN 버스 오프 상태 지속 시간 카운
☐ \ap_canComm	void CC_CanBusOffSt	002	CAN 버스 오프 상태 (CanBusOffSt

< Prev Ok Cancel

- OK
 - 선택된 TC 상세 정보 요청
 - 함수 spec, 테스트 케이스 detail
- 예외 처리
 - V-SPICE 응답에 따라 실패하는 경우 실패 코드에 경고 다이얼로그 표시

5. 선택된 TestCase 상세 정보 전송

01 CT-API 설계

SURESOFT

CT측 요구사항

사용자 시나리오 분석

Generate Code-Based Tests with AI

테스트 생성 방법

☐ 구조 기반

Goal:

☐ 요구사항 분석 기반

☒ V-SPICE 명세 기반

URL:

UserName:

Password:

- V-SPICE 명세 기반 생성을 선택하여 V-SPICE 정보 입력
 - URL
 - 사용자
 - 암호
- [Next >] 선택 시 Design 목록 요청

1. V-SPICE 로그인 정보 입력

No 1. 로그인 및 디자인 목록 반환 API

2. 로그인 성공 Design 목록 로드 및 선택 가능

Generate Code-Based Tests with AI

Design:

File:

Function:

File	Function	Test case	Description
☒ \ap_canComm	void CC_CanBusOffSt	001	CAN 버스 오프 상태 지속 카운
☐ \ap_canComm	void CC_CanBusOffSt	002	CAN 버스 오프 상태 (CanBusOffSt

- Design
 - 기본값: 빈값
 - Select: 선택된 Design 데이터 요청
 - 소스 파일, 함수, TC ID, TC Description (CT UI에 표시되는 정보)
- File/Function
 - Design이 선택되면 기본값으로 자동 설정
 - 기본값: All

3. 해당 Design의 File과 그 File의 함수 선택 가능 (기본값 All/All) ⇒ Test Case 목록 로드

No 2. 해당 디자인의 File과 Function 그리고 TC 목록을 반환하는 API

No 3. 필요한 TC를 선택하고, 해당 TC의 실제 정보를 반환하는 API

4. 필요한 Test Case들 선택

Generate Code-Based Tests with AI

Design:

File:

Function:

filter:

File	Function	Test case	Description
☒ \ap_canComm	void CC_CanBusOffSt	001	CAN 버스 오프 상태 지속 카운
☐ \ap_canComm	void CC_CanBusOffSt	002	CAN 버스 오프 상태 (CanBusOffSt

- OK
 - 선택된 TC 상세 정보 요청
 - 함수 spec, 테스트 케이스 detail
- 예외 처리
 - V-SPICE 응답에 따라 실패하는 경우 실패 코드에 경고 다이얼로그 표시

5. 선택된 TestCase 상세 정보 전송

총 3개의 API 필요 !

API 명세서 작성

분석한 사용자 시나리오 및 요구사항 기반으로 API 명세서 먼저 작성 완료

No 1. 로그인 및 디자인 목록 반환 API



Design 목록 조회 API

카테고리	CT 연동
설명	V-SPICE 접속 정보를 받아 연결 확인 후 Design 목록 반환
Method	POST
URL	/vspice/CtTestCaseControl/getDesignList
param	비어 있음
기타	세션 인증 전달
사용자	관리자
속성 추가	

No 2. 해당 디자인의 File과 Function 그리고 TC 목록을 반환하는 API



해당 디자인 상세 정보 조회 API

카테고리	CT 연동
설명	선택된 Design의 File, Function, 모든 TestCase 목록을 반환
Method	GET
URL	/vspice/CtTestCaseControl/getFileFunctionTcLists?designName={designName}
param	designName
기타	File / Function 필터링은 클라이언트에서 수행한다.
사용자	관리자

No 3. 필요한 TC를 선택하고, 해당 TC의 실제 정보를 반환하는 API



선택된 TC의 상세 정보 리턴 API

카테고리	CT 연동
설명	사용자가 선택한 Test Case를 기준으로 V-SPICE의 함수 명세 및 TC 상세 정보를 수집하여 CT 테스트 생성 파이프라인으로 전달.
Method	POST
URL	/vspice/CtTestCaseControl/getDetailTcData
param	비어 있음
기타	실제 함수 명세 및 TC 전달 파이프라인 API
사용자	관리자

01 CT-API 설계

API 명세서 작성

먼저 URL 패턴 확정

맨 처음 설계한 REST API 기반 URL 패턴

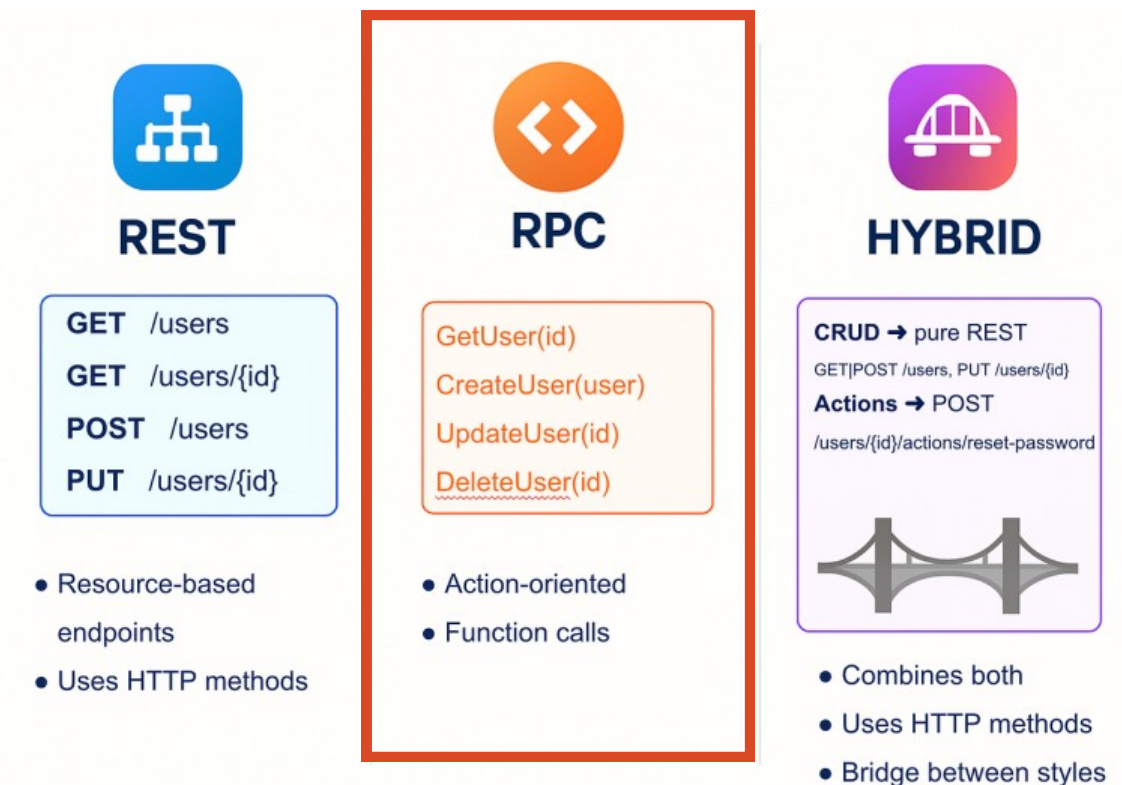
```
POST /vspice/designs
GET /vspice/designs/{designName}/testcases
POST /vspice/designs/{designName}/testcases/detail
```

“REST”

GET	/movies	Get list of movies
GET	/movies/{id}	Find a movie by its ID
POST	/movies	Create a new movie
PUT	/movies	Update an existing movie
DELETE	/movies	Delete an existing movie

리소스(Resource, 자원) 중심으로 URL을 설계.
보통 명사형 자원이 들어가고, 어떤 동작을 할지는 HTTP Method로 구분.

현재 프로젝트에서 사용하는 방식



Action(기능) 중심으로 API를 설계하는 기법
서비스 지향적, 액션과 동사에 초점

API 명세서 작성

먼저 URL 패턴 확정



RPC

GetUser(id)
CreateUser(user)
UpdateUser(id)
DeleteUser(id)

- Action-oriented
- Function calls

Action(기능) 중심으로 API를 설계하는 **RPC**의 장점

1. 복잡한 업무 기능을 더 직접적으로 표현

URL만 봐도 어떤 기능인지 바로 알 수 있다.

2. 기능 의도가 명확함

단순 CRUD가 아닌 가공 조회, 조합 조회, 상세 처리에 적합하다.

3. 설계가 직선적임

리소스 모델링에 억지로 맞추지 않고, 실제 기능 단위로 바로 엔드포인트를 만들 수 있다.

4. 내부 시스템에 실용적임

협업, 유지보수, 디버깅 시 API 의도를 빠르게 파악할 수 있다.

01 CT-API 설계

SURESOFT

API 명세서 작성

먼저 URL 패턴 확정

RPC 패턴은 Spring MVC와 특히 궁합이 좋다.

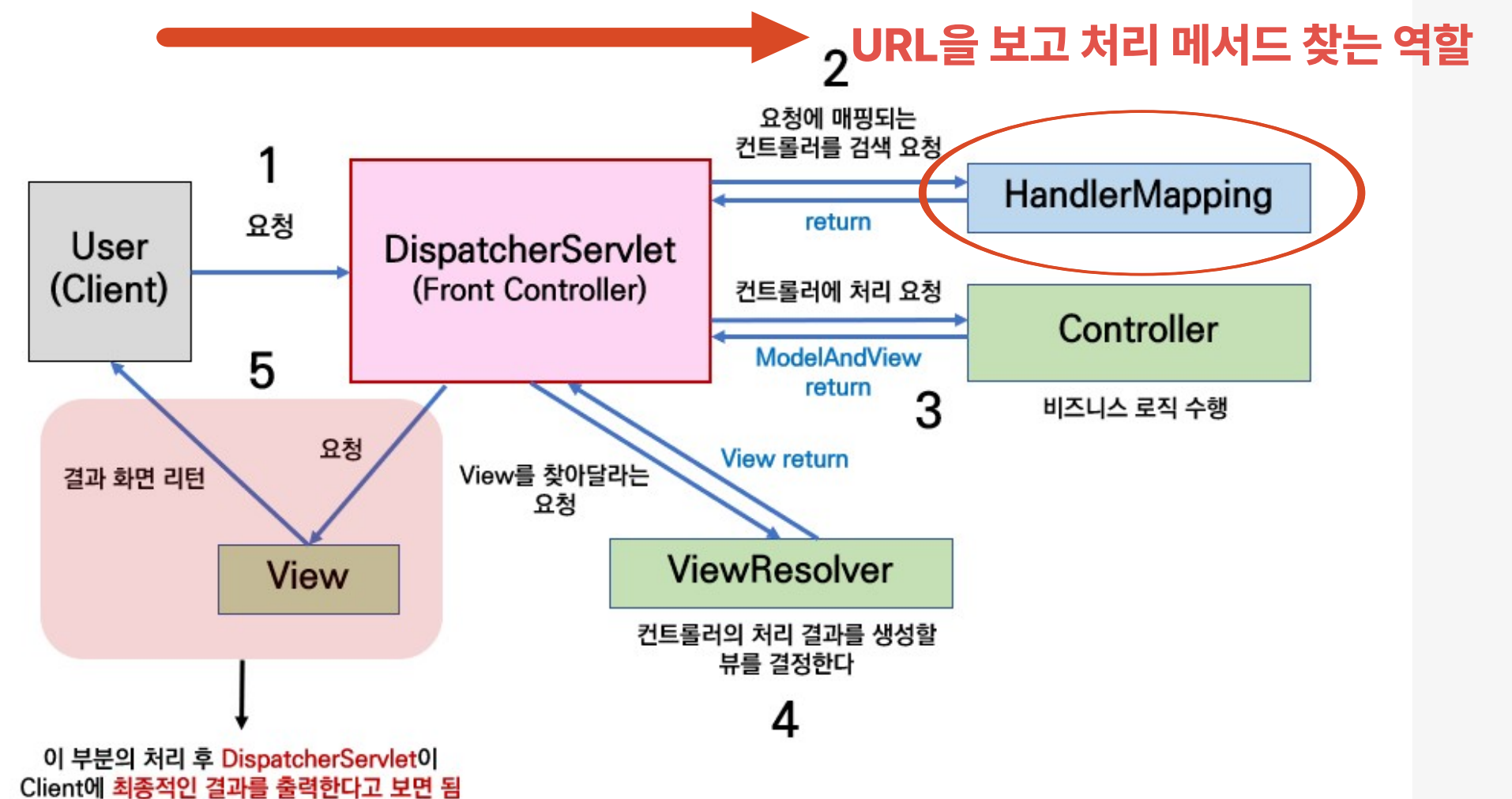
URL 이름과 컨트롤러 메서드 역할이 거의 그대로 대응

/getDesignList → getDesignList()

/getDetailTcData → getDetailTcData()

RPC 패턴식 URL의 원격 함수 호출 성격과

HandlerMapping 역할 자체가 매우 자연스럽게 이어짐



API 명세서 작성

먼저 URL 패턴 확정

RPC 패턴에 맞춰 URL 재구성 후 확정

POST /vspice/CtTestCaseControl/getDesignList

GET /vspice/CtTestCaseControl/getFileFunctionTcLists?designName={designName}

POST /vspice/CtTestCaseControl/getDetailTcData

서비스 지향적, 액션과 동사에 초점

01 CT-API 설계

SURESOFT

API 명세서 작성

Request & Response Body

필수 데이터만 우선 정의하여 JSON Request/Response 초안 작성

Function Information					
SW Unit ID	SWU_1				
Prototype	void find_state(char (*map)[maxlink * tail]				
File	\miro_func.c				
Return	void				
Description	미로 탐색 과정에서 사용할 연결 리스트를 생성하고 시작점을 찾아 목표 정보를 저장합니다. 탐색에 필요한 반복 구조를 설정하여 미로 내 초기 위치를 감지하고 노드에 상태 정보를 기록합니다. 연결 리스트와 반복 루프를 통해 탐색 준비를 완료합니다.				
Called Function	int main()				
Calling Function	link Q_make_findQ(struct link* tail, int num)				
ASIL					
Related Architecture ID					

Function Variable Info					
Input/Output	Property	Name	Data Type	Description	Scope
Input	function	Q_make_findQ(tail, 1)	link	탐색용 연결 리스트 생성 함수	
Input	parameter	tail	struct link*	연결 리스트의 꼬리 포인터	
-	local	node	struct link*	미로 탐색 상태를 저장하는 연결 리스트 노드	
-	local	count1	int	미로의 행을 순회하는 루프 카운터	-2147483648~2147483647
-	local	node->miro_x	int	미로 내 시작점의 x 좌표 저장	-2147483648~2147483647
-	local	node->miro_y	int	미로 내 시작점의 y 좌표 저장	-2147483648~2147483647
-	local	node->check	char	노드 상태를 확인하는 문자	-128~127
-	local	count2	int	미로의 열을 순회하는 루프 카운터	-2147483648~2147483647
-	unknown	map	unknown	미로 정보를 저장하는 2차원 문자 배열	

Function Specification	
1. 미로 탐색 상태를 저장하는 연결 리스트 노드(node)를 선언해야 합니다. 2. 미로의 행을 순회하는 루프 카운터(count1)와 미로의 열을 순회하는 루프 카운터(count2)를 선언해야 합니다. 3. 탐색용 연결 리스트 생성 함수(Q_make_findQ(tail, 1))를 수행해야 합니다. 4. count1을 0부터 15까지 1씩 증가시키면서 반복.	

No 2. 해당 디자인의 File과 Function
그리고 TC 목록을 반환하는 API

```
{
  "result": true,
  "resultData": {
    "files": [
      {
        "filePath": "ap_canComm.c",
        "functions": [
          {
            "functionSignature": "void CC_CanBusOffState(void)",
            "testCases": [
              {
                "testCaseCode": "001",
                "description": "CAN 버스 오프 상태 진입 시 상태 변경이 올바르게 수행된다."
              },
              {
                "testCaseCode": "002",
                "description": "CAN 버스 오프 상태가 지형 시간 이상 유지되면 커튼이"
              }
            ]
          },
          {
            "functionSignature": "void CC_RecoverBus(void)",
            "testCases": []
          }
        ]
      },
      {
        "filePath": "ap_motor.c",
        "functions": [
          {
            "functionSignature": "void MC_InitMotor(void)",
            "testCases": [
              {
                "testCaseCode": "001",
                "description": "모터 초기화 후 모터 초기화 후속이 정상 수행된다."
              }
            ]
          },
          {
            "functionSignature": "void MC_StopMotor(void)",
            "testCases": []
          }
        ]
      }
    ]
  },
  "resultMessage": "디자인 설계 정보 조회 성공"
}
```

No 3. 필요한 TC를 선택하고,
해당 TC의 실제 정보를 반환하는 API

```
{
  "result": true,
  "resultData": {
    "functions": [
      {
        "filePath": "/ap_canComm.c",
        "prototype": "void CC_CanBusOffState(uint8 channel)",
        "funcSpec": "1. (node) (tail->prev->prev)로 이동하여 링크D.Vn2. (cross  
"info": [
          {
            "direction": "Input",
            "property": "function",
            "name": "map_out(moving)",
            "description": "미로 내 상태를 저장하는 2차원 문자 배열.",
            "scope": "-2147483648~2147483647"
          },
          {
            "direction": "none",
            "property": "local",
            "name": "node->prev",
            "description": "링크D 링크D의 이전 링크를 포인터 변수.",
            "scope": "-2147483648~2147483647"
          }
        ],
        "caller_list": ["int main()"],
        "callee_list": ["struct link Q_make()", "void map_out()"],
        "testCases": [
          {
            "tcid": "001",
            "description": "CAN Bus Off 상태 진입 시 오프 상태가 올바르게 처리된다.",
            "values": [
              {
                "valueName": "channel",
                "valueType": "Input",
                "value": "1"
              },
              {
                "valueName": "g_canBusOffState[1]",
                "valueType": "Expected",
                "value": "1"
              }
            ]
          },
          {
            "tcid": "002",
            "description": "CAN Bus Off 상태 해제 시 상태 값이 정상으로 복구되어야",
            "values": [
              {
                "valueName": "channel",
                "valueType": "Input",
                "value": "1"
              },
              {
                "valueName": "g_canBusOffState[1]",
                "valueType": "Expected",
                "value": "0"
              }
            ]
          }
        ]
      }
    ]
  },
  "resultMessage": "선택된 TC들의 실제 정보 조회 성공"
}
```

Nested / Flatten 두 가지 버전으로 제공
⇒ CT측 협의 후 Nested 구조로 최종 확정

01 CT-API 설계

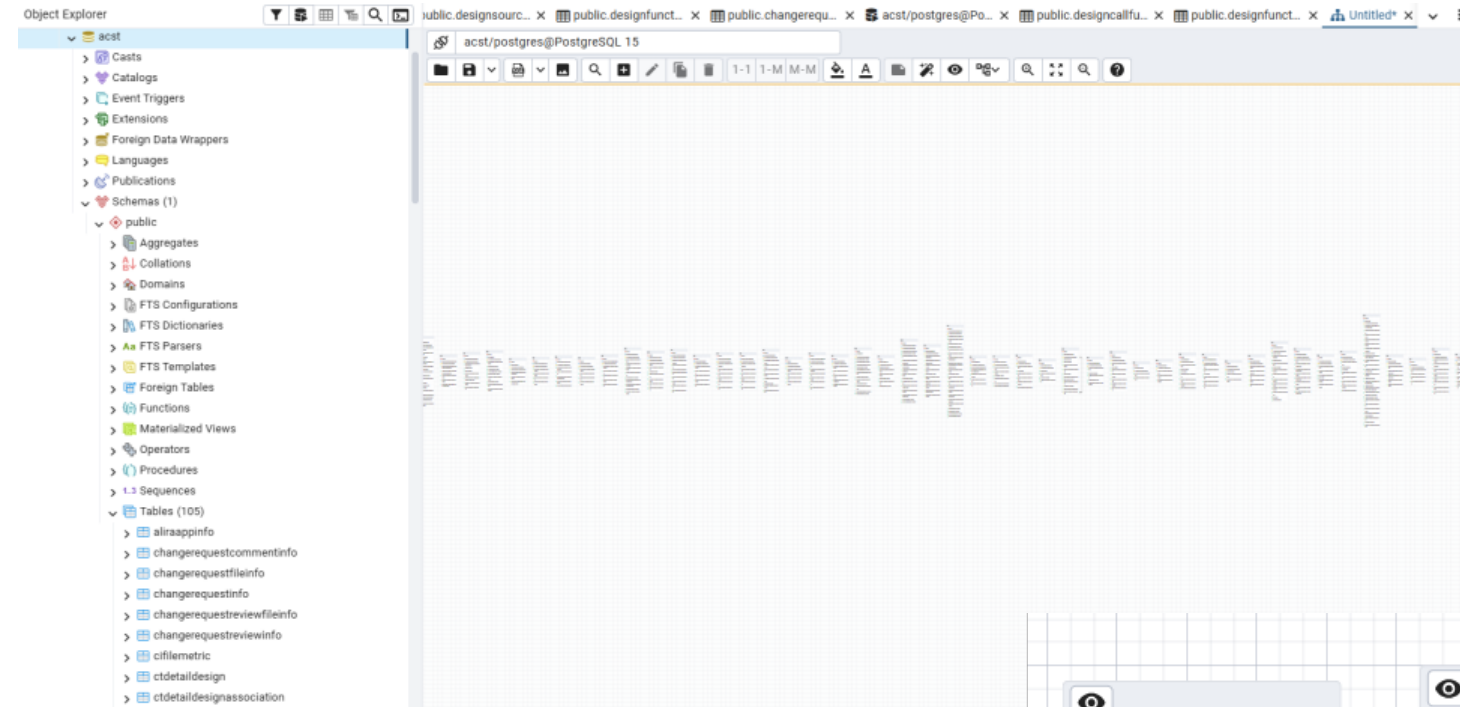
SURESOFT

API 명세서 작성

Request & Response Body

내가 봐야하는 DB TABLE들

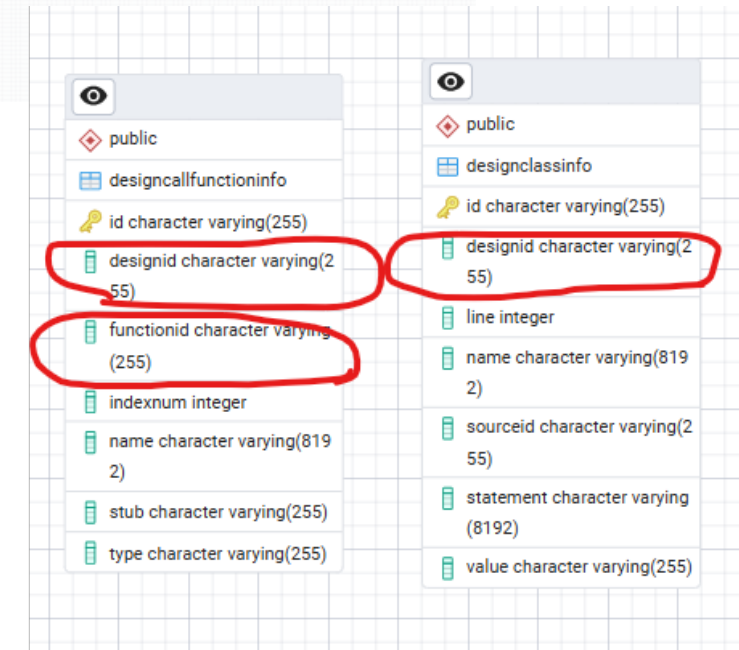
designfunctioninfo => 함수 명세 및 경로
designfunctiontestcase => 테스트케이스 valuetype: input이랑 expect만 내보내면됨
designioinfo => 함수의 input output 정보 (함수의 variable info)
designsourceinfo => 파일의 이름과 path 정보



DB 구조 분석 과정에서 대부분의 테이블이 FK(외래키) 제약 없이 설계된 것을 확인

테이블에 연관 테이블의 id 값이 일반 컬럼으로 들어있음

→ 이걸 기반으로 서비스 로직이나 SQL join으로 직접 연결해서 사용하는 구조



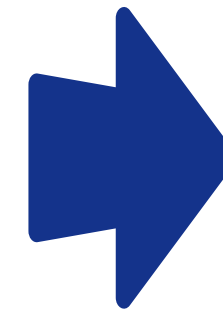
FK 강제 X, 논리적으로만 연결!

API 명세서 작성

Request & Response Body

장점

- 1) 참조 무결성 검사를 스킵함으로써 성능 부담을 줄일 수 있음
→ 대량 작업 및 적재 작업에서 가볍게 동작
- 2) 스키마 변경과 데이터 이관이 유연함
→ 데이터 마이그레이션과 외부 시스템 연동에 유연한 대응
- 3) 시스템 간 결합도를 낮출 수 있음
→ 모듈 간 독립성 유지, 필요 시에만 서비스 단에서 조합



FK-less Schema 구조

관계는 존재하지만, 그 관계를 DB가 강제하지 않고
쿼리와 서비스 로직에서 직접 관리

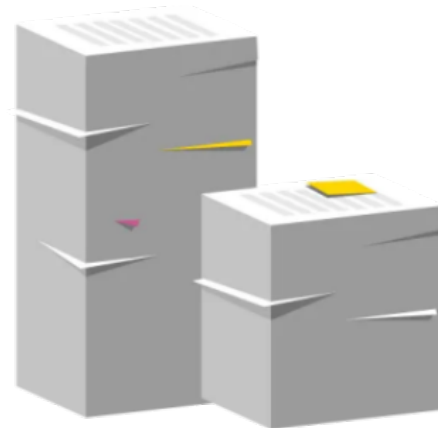
주로 어디서 쓰이나

- 대용량 서비스 / 쓰기 부하가 큰 시스템
- MSA나 멀티 모듈 환경
- 복잡한 업무 시스템
- 다양한 데이터를 한데 모으는 통합형 시스템

DB가 참조 무결성을 보장해주지 않으니, "이 값이 실제로 유효한 관계인가?" 를 더 조심해서 보가면서 작업 진행

API 명세서 작성

고려사항



실제 고객사 환경에서는 Design별 파일 수가 수천~수만 건,
이에 연결된 **Function/Test Case**도 대규모일 수 있다고 가정

초기 TC 리스트 조회 구간에서 대용량 데이터를
어떤 방식으로 완충할지 중점적으로 검토

Generate Code-Based Tests with AI

Design:

File:

Function:

filter...

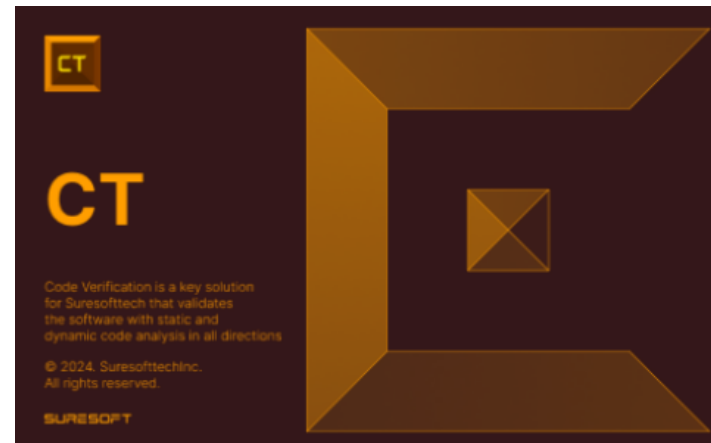
	File	Function	Test case	Description
☒	\ap_canComn	void CC_CanBusOffSt	001	CAN 버스 오프 상태 지속 시간 카운
☐	\ap_canComn	void CC_CanBusOffSt	002	CAN 버스 오프 상태(l_canBusOffSt

< Prev Ok Cancel

- 단계적 조회 / 점진적 조회
- 커서 페이지네이션 고려

API 명세서 작성

고려사항



다만 CT 측 요구사항은 All / All 기본값 기반의
전체 데이터 일괄 조회 후 클라이언트 필터링 방식

이에 맞춰, 대용량 데이터를 한 번에 내려보내는 상황을 전제로
응답 구조와 조회 성능 최적화 전략을 집중적으로 검토



추후 json 용량이 너무 커지면
gzip이나 brotli 같은 json 압축 기법을 도입하면 어떨까도 구상

이렇게 명세 및 설계 구상 완료 후 실제 개발 시작

02 CT-API 구현 및 개선

SURESOFT

API 구현

기본 구조 구현

No 2. 해당 디자인의 File과 Function 그리고 TC 목록을 반환하는 API



해당 디자인 상세 정보 조회 API

카테고리	CT 연동
설명	선택된 Design의 File, Function, 모든 TestCase 목록을 반환
Method	GET
URL	/vspice/CtTestCaseControl/getFileFunctionTcLists?designName={designName}
param	designName
기타	File / Function 필터링은 클라이언트에서 수행한다.
사용자	관리자

No 3. 필요한 TC를 선택하고, 해당 TC의 실제 정보를 반환하는 API



선택된 TC의 상세 정보 리턴 API

카테고리	CT 연동
설명	사용자가 선택한 Test Case를 기준으로 V-SPICE의 함수 명세 및 TC 상세 정보를 수집하여 CT 테스트 생성 파이프라인으로 전달.
Method	POST
URL	/vspice/CtTestCaseControl/getDetailTcData
param	비어 있음
기타	실제 함수 명세 및 TC 전달 파이프라인 API
사용자	관리자

No 1. API는 로그인 로직 때문에 제일 마지막에 작업

먼저 요청과 응답 스펙에 맞는 기본 기능을 Lean하게 완성.
초기 단계에서는 정확한 동작과 API 명세 만족을 우선 목표로 설정.

02 CT-API 구현 및 개선

API 구현

기본 구조 구현

DTO + Builder 패턴 적용

명확한 응답 설계와 안정적인 객체 생성 보장

DB (flat rows)

↓ Repository (FlatRow interface projection)

↓ Service (Pass Pattern으로 계층 조립)

↓ Builder로 중첩 DTO 생성

Controller → JSON 직렬화 → 외부 응답

✓ 딱 필요한 JOIN 쿼리 결과 매핑용
Projection Interface

```
// --- 쿼리 결과 매핑용 Row ---  
  
// native query projection interface: SQL alias → getter명 매핑 (case-insensitive)  
public interface FunctionInfoRow { 6개 사용 위치 ⚠.kim  
    String getFilePath(); ⚠.kim  
    String getFunctionId(); ⚠.kim  
    String getSignature(); ⚠.kim  
    String getSpec(); ⚠.kim  
}  
  
@Getter 3개 사용 위치 ⚠.kim  
@AllArgsConstructor  
public static class IoInfoRow {  
    private final String functionId;  
    private final String direction;  
    private final String property;  
    private final String name;  
    private final String description;  
    private final String scope;  
}  
  
// native query projection interface: SQL alias → getter명 매핑 (case-insensitive)  
public interface TcValueRow { 3개 사용 위치 ⚠.kim  
    String getFunctionId(); ⚠.kim  
    String getTcId(); ⚠.kim  
    String getValueType(); 2개 사용 위치 ⚠.kim  
    String getValueName(); 1개 사용 위치 ⚠.kim  
    String getValue(); ⚠.kim  
}
```

✓ 재사용성보다 명확성 선택한 DTO

```
package com.suresoft.qscroll.acst.jdbc.domain;  
  
import java.util.List;  
  
@Getter 13개 사용 위치 ⚠.kim  
@Builder  
@NoArgsConstructor  
@AllArgsConstructor  
public class CtDetailTcDataResponse {  
  
    private List<FunctionEntry> functions;  
    @JsonIgnore  
    private List<String> notFound;  
  
    @Getter 3개 사용 위치 ⚠.kim  
    @Builder  
    @NoArgsConstructor  
    @AllArgsConstructor  
    public static class FunctionEntry {  
        private String filePath;  
        private String prototype;  
        private String funcSpec;  
        private List<IoInfo> ioInfo;  
        private List<TcEntry> testCases;  
    }  
  
    @Getter 2개 사용 위치 ⚠.kim  
    @Builder  
    @NoArgsConstructor  
    @AllArgsConstructor  
    public static class IoInfo {  
        private String direction;  
        private String property;  
        private String name;  
        private String description;  
        private String scope;  
    }  
  
    @Getter 7개 사용 위치 ⚠.kim  
    @Builder  
    @NoArgsConstructor  
    @AllArgsConstructor  
    public static class TcEntry {  
        private String tcId;  
        private String description;  
        private List<TcValue> values;  
    }  
  
    @Getter 4개 사용 위치 ⚠.kim  
    @Builder  
    @NoArgsConstructor  
    @AllArgsConstructor  
    public static class TcValue {  
        private String valueName;  
        private String valueType;  
        private String value;  
    }  
}
```

```
package com.suresoft.qscroll.acst.jdbc.domain;  
  
import java.util.List;  
  
@Getter 20개 사용 위치 ⚠.kim  
@Builder  
@NoArgsConstructor  
@AllArgsConstructor  
public class CtFileFunctionCtListsResponse {  
  
    private List<FileEntry> files;  
  
    @Getter 4개 사용 위치 ⚠.kim  
    @Builder  
    @NoArgsConstructor  
    @AllArgsConstructor  
    public static class FileEntry {  
        private String filePath;  
        private List<FunctionEntry> functions;  
    }  
  
    @Getter 1개 사용 위치 ⚠.kim  
    @Builder  
    @NoArgsConstructor  
    @AllArgsConstructor  
    public static class FunctionEntry {  
        private String functionSignature;  
        private List<TcEntry> testCases;  
    }  
  
    @Getter 4개 사용 위치 ⚠.kim  
    @Builder  
    @NoArgsConstructor  
    @AllArgsConstructor  
    public static class TcEntry {  
        private String testCaseCode;  
        private String description;  
    }  
  
    // native query flat projection  
    public interface FlatRow { 3개 사용 위치 ⚠.kim  
        String getFilePath(); ⚠.kim  
        String getFunctionId(); ⚠.kim  
        String getFunctionSignature(); ⚠.kim  
        String getTestCaseCode(); 1개 사용 위치 ⚠.kim  
        String getDescription(); ⚠.kim  
    }  
}
```


02 CT-API 구현 및 개선

SURESOFT

API 구현

기본 구조 구현

No 1. 로그인 및 디자인 목록 반환 API 구현

Spring Security 기반으로 인증 수행



인증 성공 시 세션 등록



세션 ID 생성 및 발급



이후 JSESSIONID 및 디자인 목록 반환

```
// Design 목록 조회 API
// username/password 인증 후 세션 등록, 전체 Design 목록 반환
@ResponseBody & jkim
@RequestMapping(value = "/getDesignList", method = RequestMethod.POST)
public RetResult getDesignList(@RequestBody CtDesignListRequest request, HttpServletRequest httpRequest) {
    RetResult re = new RetResult();

    Authentication authentication;
    try {
        authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(request.getUsername(), request.getPassword()));
    } catch (Exception e) {
        re.setResult(false);
        re.setResultData("");
        re.setResultMessage("V-SPICE 인증 실패: " + e.getMessage());
        return re;
    }

    SecurityContextHolder.getContext().setAuthentication(authentication);

    String jsessionId = httpRequest.getSession(true).getId();
    CtDesignListResponse response = ctTestCaseService.getDesignList(jsessionId);
    re.setResult(true);
    re.setResultData(response);
    re.setResultMessage("Design 목록 조회 성공");
    return re;
}
```

로그인 API라 인증 필요 페이지에서 제외 →

```
// 상단 Root 페이지를 제외한 모든 페이지 인증 필요페이지로 설정
http.authorizeRequests(ExpressionInterceptUrlRegistry requests -> requests.antMatchers(
    // Allow Url
    .antMatchers(antMatchers: "/login",
        "/LoginControll/saveUser",
        "/LoginControll/showUser",
        "/ProjectControll/projectListAll",
        "/ProjectControll/projectInfo",
        "/LicenseControll/license",
        "/UserRegister",
        "/api/authenticate",
        "/CtTestCaseControl/getDesignList").permitAll());
```

02 CT-API 구현 및 개선

SURESOFT

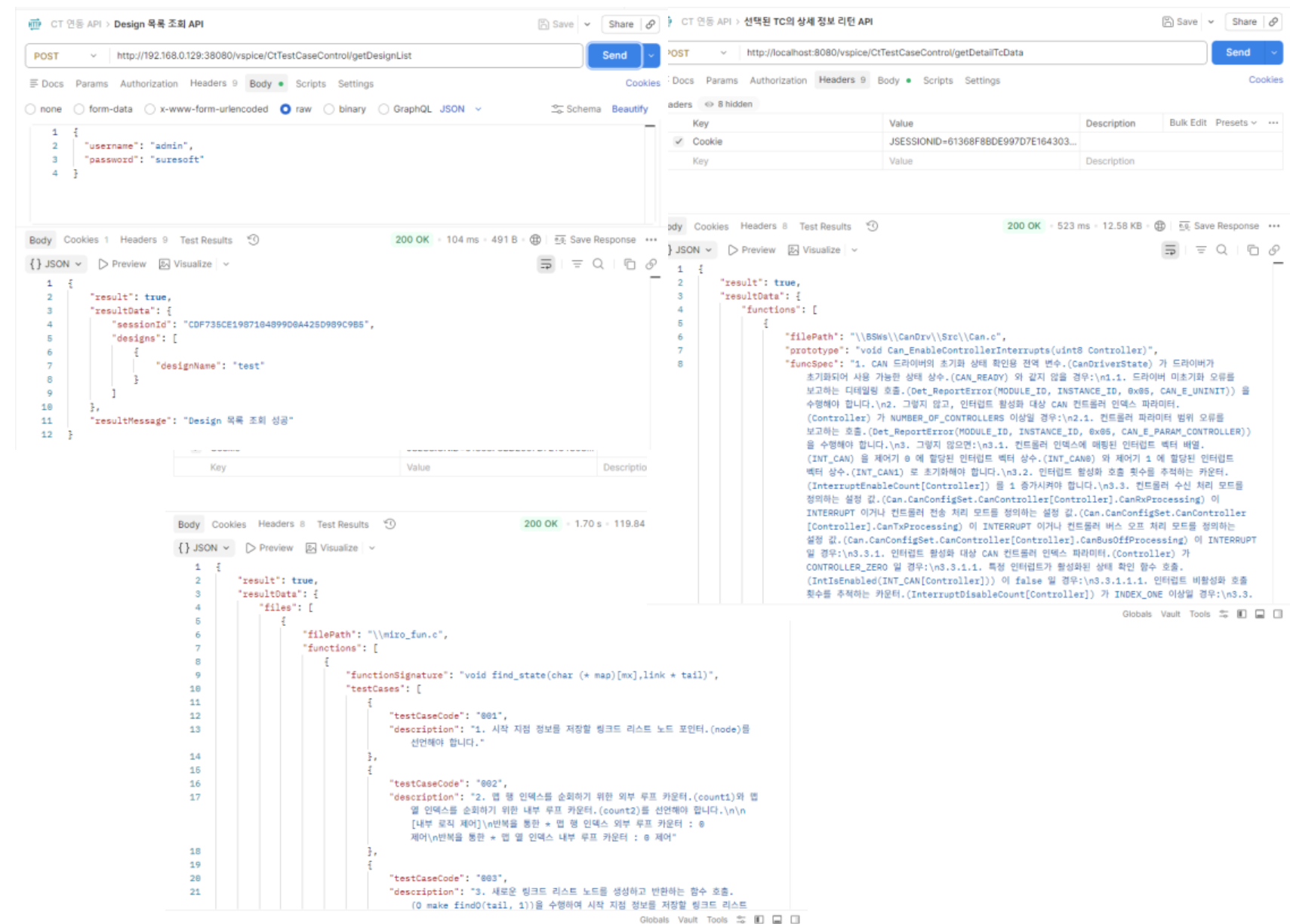
API 구현

기본 구조 구현 완료

Controller - Service - Repository 계층으로 3 API의 기본 흐름 구현 후 정상 동작 확인

요청 명세에 맞는 형식으로 응답이 정상 생성됨을 확인

대규모 조회를 하는 No2. No3. API를 중점적으로 이후 병목 구간을 분석하여 성능 최적화 리팩토링 진행



API 성능 최적화 리팩토링

주요 개선 포인트

대용량 데이터 조회 구조에서
N+1 문제 및 쿼리 폭증 가능성을 사전 식별



DB → Service → JSON 직렬화
전 구간에서 병목 발생 가능성 고려

1) 조회 횟수 최적화

- N+1 문제 제거
- DB round trip 최소화
- 분산된 조회를 통합하여 불필요한 쿼리 반복 방지

2) 조회 데이터 양 최적화

- DB 단계에서 유효한 데이터만 선별하도록 쿼리 재설계
- TC가 존재하는 함수만 조회하도록 필터링
- DB 필터링을 활용해 불필요한 row를 사전에 제거

3) 서비스 단 가공 최적화

- Map 기반 다중 구조 대신 정렬된 flat 데이터 기반 순차 조립 방식 적용
- 이중 반복문 제거
- 중간 자료구조 최소화로 JVM 메모리 사용량과 가공 비용 절감

02 CT-API 구현 및 개선

SURESOFT

API 성능 최적화 리팩토링

1. 조회 횟수 최적화

No 2. 해당 디자인의 File과 Function 그리고 TC 목록을 반환하는 API

```
// 최종 응답 조합
// CtFileFunctionTcListsResponse.FileEntry: filePath + FunctionEntry 리스트
List<CtFileFunctionTcListsResponse.FileEntry> files = new ArrayList<>();
for (Map.Entry<String, List<Object[]>> sourceEntry : sourceMap.entrySet()) {
    String filePath = sourceEntry.getKey();
    // CtFileFunctionTcListsResponse.FunctionEntry: functionSignature + TcEntry 리스트
    List<CtFileFunctionTcListsResponse.FunctionEntry> functionEntries = new ArrayList<>();

    for (Object[] funcRow : sourceEntry.getValue()) {
        String functionId = (String) funcRow[1];
        String signature = (String) funcRow[2];

        Map<String, TcAssembly> tcsByFunction =
            tcMap.getOrDefault(functionId, Collections.emptyMap());

        // input/expect가 하나라도 있는 TC만 포함
        // CtFileFunctionTcListsResponse.TcEntry: testCaseCode(TC ID) + description
        List<CtFileFunctionTcListsResponse.TcEntry> tcEntries = tcsByFunction.entrySet().stream()
            .filter(e -> e.getValue().hasInputOrExpect())
            .map(e -> CtFileFunctionTcListsResponse.TcEntry.builder()
                .testCaseCode(e.getKey())
                .description(e.getValue().description)
                .build())
            .collect(Collectors.toList());

        if (tcEntries.isEmpty()) continue;

        functionEntries.add(CtFileFunctionTcListsResponse.FunctionEntry.builder()
            .functionSignature(signature)
            .testCases(tcEntries) // TcEntry 리스트
            .build());
    }
}
```

```
public CtFileFunctionTcListsResponse getFileFunctionTcLists(String designName) {
    // 쿼리 1: designName → designId
    DesignInfo design = designInfoRepository.findByDesignName(designName);
    if (design == null) {
        return null;
    }
    String designId = design.getId();

    // 쿼리 2: 소스파일 + 함수 한 번에 (path ASC, name ASC 정렬)
    // row: [0]=filePath, [1]=functionId, [2]=functionSignature
    List<Object[]> sourceFunctionRows = ctTestCaseQueryRepository.findSourceFunctionsByDesignId(designId);

    // 쿼리 3: 해당 design의 TC 데이터 전체 일괄 조회
    // row: [0]=functionId, [1]=tcId, [2]=valueType, [3]=value
    List<Object[]> tcRows = ctTestCaseQueryRepository.findTcDataByDesignId(designId);
}
```

BEFORE

- designName → designId를 먼저 별도 조회
- 이후 source/function 조회, TC 조회를 순차적으로 분리 실행
- 총 3번의 DB round-trip 발생
- 각 쿼리 결과를 Java 서비스 단에서 다시 조합해야 했음

02 CT-API 구현 및 개선

SURESOFT

API 성능 최적화 리팩토링

1. 조회 횟수 최적화 No 2. 해당 디자인의 File과 Function 그리고 TC 목록을 반환하는 API

```
// CTE 1-query: valid TC가 있는 함수의 flat rows로 반환
// ORDER BY s.path, f.name, f.id, ta.tcid → 서비스 순차 pass 패턴 견제
// alias → FlatRow getter: filePath/functionId/functionSignature/testCaseCode/description
@Query(nativeQuery = true, value = """
    WITH di AS (
        SELECT id FROM designinfo WHERE name = :designName
    ),
    tc_agg AS (
        SELECT d.functionid, d.tcid,
        MAX(CASE WHEN d.valuetype = 'description' THEN d.value END) AS description
        FROM designfunctiontestcase d
        WHERE d.designid = (SELECT id FROM di)
        GROUP BY d.functionid, d.tcid
    )
    SELECT s.path AS filePath,
    f.id AS functionId,
    f.signature AS functionSignature,
    ta.tcid AS testCaseCode,
    ta.description AS description
    FROM di
    JOIN designsourceinfo s ON s.designid = di.id AND s.isheader = false
    JOIN designfunctioninfo f ON f.sourceid = s.id AND f.designid = di.id AND f.active = true
    LEFT JOIN tc_agg ta ON ta.functionid = f.id
    ORDER BY s.path ASC, f.name ASC, f.id ASC, ta.tcid ASC NULLS FIRST
""")
List<CtFileFunctionTcListsResponse.FlatRow> findFlatByDesignName(@Param("designName") String designName);
```

```
public CtFileFunctionTcListsResponse getFileFunctionTcLists(String designName) {
    // findFlatByDesignName: CTE 1개 쿼리로 file/function/TC를 flat rows로 반환
    // ORDER BY 보장: s.path ASC, f.name ASC, f.id ASC, ta.tcid ASC
    List<CtFileFunctionTcListsResponse.FlatRow> flatRows =
        ctTestCaseQueryRepository.findFlatByDesignName(designName);

    if (flatRows.isEmpty()) {
        // design 존재 여부로 원인 구분
        if (designInfoRepository.findByName(designName) == null) {
            return null; // design 없음
        }
        return CtFileFunctionTcListsResponse.builder().files(new ArrayList<>()).build(); // valid TC 없음
    }
}
```

AFTER

- designName 기준 조회와 유효 TC 필터링을 서브쿼리 기반 단일 쿼리로 통합
- designId를 Java에서 별도로 조회·전달하지 않고, 쿼리 내부에서 직접 처리
- 총 1번의 DB round-trip으로 축소
- 서비스 단의 중간 조립 부담 감소

⇒ 가공된 결과 조회에 더 적합한 Native Query를 사용해 flat한 형태로 일괄 조회. 추후 서비스 단에서 가공

02 CT-API 구현 및 개선

SURESOFT

API 성능 최적화 리팩토링

1. 조회 횟수 최적화 No 3. 필요한 TC를 선택하고, 해당 TC의 실제 정보를 반환하는 API

```
////////// 선택된 TC의 상세 정보 리턴 API ////////////

// 선택된 (filePath, signature) 조합에 해당하는 함수 정보 일괄 조회 (getDetailTcData를)
// 반환: [0]=filePath, [1]=functionId, [2]=signature, [3]=spec
@Query("SELECT s.path, f.id, f.signature, f.spec " +
    "FROM designsourceinfo s, designfunctioninfo f " +
    "WHERE s.designId = :designId AND f.designId = :designId " +
    "AND f.sourceId = s.id " +
    "AND s.path IN :filePaths AND f.signature IN :signatures")
List<Object[]> findFunctionsByDesignIdAndPathsAndSignatures(
    @Param("designId") String designId,
    @Param("filePaths") List<String> filePaths,
    @Param("signatures") List<String> signatures);

// 선택된 functionId 목록의 IO 정보 일괄 조회 (getDetailTcData를)
// 반환: [0]=functionId, [1]=direction, [2]=property, [3]=name, [4]=description, [5]=scope
@Query("SELECT io.functionId, io.direction, io.property, io.name, io.description, io.scope " +
    "FROM designioinfo io " +
    "WHERE io.functionId IN :functionIds " +
    "ORDER BY io.functionId ASC, " +
    "CASE io.direction WHEN 'both' THEN 1 WHEN 'input' THEN 2 WHEN 'output' THEN 3 WHEN 'none' THEN 4 ELSE 5 END ASC")
List<Object[]> findIoInfoByFunctionIds(@Param("functionIds") List<String> functionIds);

// 선택된 functionId + tcId 조합의 TC 값 일괄 조회 (getDetailTcData를)
// 반환: [0]=functionId, [1]=tcId, [2]=valueType, [3]=valueName, [4]=value
@Query("SELECT d.functionId, d.tcId, d.valueType, d.valueName, d.value " +
    "FROM designfunctiontestcase d " +
    "WHERE d.functionId IN :functionIds AND d.tcId IN :tcIds " +
    "AND d.valueType IN ('input', 'expect', 'description')")
List<Object[]> findTcValuesByFunctionIdsAndTcIds(
    @Param("functionIds") List<String> functionIds,
    @Param("tcIds") List<String> tcIds);
```

```
// 루틴 3: IO 정보 일괄 조회
// row: [0]=functionId, [1]=direction, [2]=property, [3]=name, [4]=description, [5]=scope
List<Object[]> ioRows = ctTestCaseQueryRepository.findIoInfoByFunctionIds(functionIds);

// CtDetailTcDataResponse.IoInfo: 함수의 IO 관련 정보 (direction, property, name, description, scope)
Map<String, List<CtDetailTcDataResponse.IoInfo>> ioMap = new LinkedHashMap<>();
for (Object[] row : ioRows) {
    String functionId = (String) row[0];
    ioMap.computeIfAbsent(functionId, k -> new ArrayList<>())
        .add(CtDetailTcDataResponse.IoInfo.builder()
            .direction((String) row[1])
            .property((String) row[2])
            .name((String) row[3])
            .description((String) row[4])
            .scope((String) row[5])
            .build());
}
```

BEFORE

- 요청 시작 시 designName → designId를 먼저 별도 조회
- 이후 function 조회, IO info 조회, TC values 조회를 순차 실행
- 총 4번의 DB round-trip 발생
- designId를 서비스 단에서 계속 전달·관리해야 했음

```
// 루틴 1: designName → designId
DesignInfo design = designInfoRepository.findByDesignName(request.getDesignName());
if (design == null) {
    return null;
}
String designId = design.getId();

List<CtDetailTcDataRequest.SelectedRow> selectedRows = request.getSelectedRows();
if (selectedRows == null || selectedRows.isEmpty()) {
    return CtDetailTcDataResponse.builder().functions(new ArrayList<>()).build();
}

// selectedRows 그룹화: Map<functionKey, List<tcCode>>
// funcKey = filePath + "\u0001" + functionSignature
Map<String, List<String>> funcToTcCodes = new LinkedHashMap<>();
for (CtDetailTcDataRequest.SelectedRow row : selectedRows) {
    String key = row.getFilePath() + "\u0001" + row.getFunctionSignature();
    funcToTcCodes.computeIfAbsent(key, k -> new ArrayList<>()).add(row.getTestCaseCode());
}
```

```
// 루틴 2: 함수 정보 일괄 조회
// row: [0]=filePath, [1]=functionId, [2]=signature, [3]=spec
List<Object[]> funcRows = ctTestCaseQueryRepository.findFunctionsByDesignIdAndPathsAndSignatures(
    designId, new ArrayList<>(filePathSet), new ArrayList<>(signatureSet));

// Map<functionKey, [functionId, spec]> 조합 (요청에 포함된 함수)
Map<String, String[]> funcKeyToInfo = new LinkedHashMap<>();
List<String> functionIds = new ArrayList<>();
for (Object[] row : funcRows) {
    String path = (String) row[0];
    String functionId = (String) row[1];
    String sig = (String) row[2];
    String spec = (String) row[3];
    String key = path + "\u0001" + sig;
    if (funcToTcCodes.containsKey(key) && !funcKeyToInfo.containsKey(key)) {
        funcKeyToInfo.put(key, new String[]{functionId, spec != null ? spec : ""});
        functionIds.add(functionId);
    }
}
```

```
// 루틴 4: TC 값 일괄 조회
// row: [0]=functionId, [1]=tcId, [2]=valueType, [3]=valueName, [4]=value
List<Object[]> tcRows = ctTestCaseQueryRepository.findTcValuesByFunctionIdsAndTcIds(
    functionIds, new ArrayList<>(tcIdSet));

// Map<functionId, Map<tcId, TcDetailAssembly>> 조합
Map<String, Map<String, TcDetailAssembly>> tcMap = new LinkedHashMap<>();
for (Object[] row : tcRows) {
    String functionId = (String) row[0];
    String tcId = (String) row[1];
    String valueType = (String) row[2];
    String valueName = (String) row[3];
    String value = (String) row[4];

    tcMap.computeIfAbsent(functionId, k -> new LinkedHashMap<>())
        .computeIfAbsent(tcId, k -> new TcDetailAssembly());

    TcDetailAssembly tc = tcMap.get(functionId).get(tcId);
    if ("description".equals(valueType)) {
        tc.description = value != null ? value : "";
    } else if ("input".equals(valueType) || "expect".equals(valueType)) {
        String responseValueType = "input".equals(valueType) ? "input" : "expected";
        // CtDetailTcDataResponse.TcValue: TC의 실제 값 (valueName, valueType(Input/Expected), value)
        tc.values.add(CtDetailTcDataResponse.TcValue.builder()
            .valueName(valueName)
            .valueType(responseValueType)
            .value(value)
            .build());
    }
}
```

API 성능 최적화 리팩토링

1. 조회 횟수 최적화

No 3. 필요한 TC를 선택하고,
해당 TC의 실제 정보를 반환하는 API

```
////////// 선택된 TC의 상세 정보 리턴 API //////////  
  
+ // Q1: function flat (designName 인라인 + round trip 제거)  
+ // JOIN designsourceinfo, designfunctioninfo + filePath/functionId/signature/spec  
+ @Query("SELECT new com.suresoft.qscroll.acst.jdbc.domain.CtDetailTcDataResponse$FunctionInfoRow(" +  
+ "s.path, f.id, f.signature, f.spec) " +  
+ "FROM designinfo di " +  
+ "JOIN designsourceinfo s ON s.designId = di.id " +  
+ "JOIN designfunctioninfo f ON f.sourceId = s.id AND f.designId = di.id" +  
+ "WHERE di.name = :designName AND s.path IN :filePaths AND f.signature IN :signatures")  
+ List<CtDetailTcDataResponse.FunctionInfoRow> findFunctionsFlatByDesignNameAndPathsAndSignatures(  
+ @Param("designName") String designName,  
+ @Param("filePaths") List<String> filePaths,  
+ @Param("signatures") List<String> signatures);  
  
+ // Q2: IO info flat  
+ // ORDER BY functionId + functionId 기준 그룹별 연속 처리 가능  
+ @Query("SELECT new com.suresoft.qscroll.acst.jdbc.domain.CtDetailTcDataResponse$IoInfoRow(" +  
+ "io.functionId, io.direction, io.property, io.name, io.description, io.scope) " +  
+ "FROM designioinfo io " +  
+ "WHERE io.functionId IN :functionIds " +  
+ "ORDER BY io.functionId ASC")  
+ List<CtDetailTcDataResponse.IoInfoRow> findIoInfoByFunctionIds(  
+ @Param("functionIds") List<String> functionIds);  
  
+ // Q3: TC values flat  
+ // ORDER BY functionId, tcId + flat composite key Map 조립  
+ @Query("SELECT new com.suresoft.qscroll.acst.jdbc.domain.CtDetailTcDataResponse$TcValueRow(" +  
+ "d.functionId, d.tcId, d.valueType, d.valueName, d.value) " +  
+ "FROM designfunctiontestcase d " +  
+ "WHERE d.functionId IN :functionIds AND d.tcId IN :tcIds " +  
+ "AND d.valueType IN ('input', 'expect', 'description')")  
+ List<CtDetailTcDataResponse.TcValueRow> findTcValuesByFunctionIdsAndTcIds(  
+ @Param("functionIds") List<String> functionIds,  
+ @Param("tcIds") List<String> tcIds);  
}
```

```
+ // Q1: function flat (designName 인라인 + round trip 제거)  
+ // JOIN designsourceinfo, designfunctioninfo + filePath/functionId/signature/spec  
+ @Query("SELECT new  
+ com.suresoft.qscroll.acst.jdbc.domain.CtDetailTcDataResponse$FunctionInfoRow(" +  
+ "s.path, f.id, f.signature, f.spec) " +  
+ "FROM designinfo di " +  
+ "JOIN designsourceinfo s ON s.designId = di.id " +  
+ "JOIN designfunctioninfo f ON f.sourceId = s.id AND f.designId = di.id " +  
+ "WHERE di.name = :designName AND s.path IN :filePaths AND f.signature IN  
+ :signatures")  
+ List<CtDetailTcDataResponse.FunctionInfoRow>  
+ findFunctionsFlatByDesignNameAndPathsAndSignatures(  
+ @Param("designName") String designName,  
+ @Param("filePaths") List<String> filePaths,  
+ @Param("signatures") List<String> signatures);
```

Inline DTO 생성

AFTER

- designName을 쿼리에 직접 전달하고, 쿼리 내부에서 JOIN으로 처리
- 사전 designId 조회 제거
- 이후 필요한 데이터만 조회하도록 단순화
총 3번의 DB round-trip으로 축소

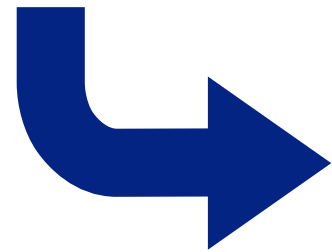
API 성능 최적화 리팩토링

2. 조회 데이터 양 최적화

No 2. 해당 디자인의 File과 Function
그리고 TC 목록을 반환하는 API

기존에는 DB에서 전체 조회 후

```
-- 전체 TC rows (~101,250개) 무조건 조회
SELECT d.functionId, d.tcId, d.valueType, d.value
FROM designfunctiontestcase d
WHERE d.designId = :designId
AND d.valueType IN ('input', 'expect', 'description')
```



```
// Java에서 유효성 판별 (전송 후 처리)
if ("input".equals(valueType) || "expect".equals(valueType)) {
    tc.hasInputOrExpect = true;
}
// ...
List<TcEntry> tcEntries = tcsByFunction.entrySet().stream()
    .filter(e -> e.getValue().hasInputOrExpect) // ← 여기서 걸러냄
    .map(...)
    .collect(...);
```

서비스 단에서 필터링

BEFORE

- 해당 design의 TC 데이터를 조건 없이 전체 조회
- 약 100k 이상의 raw row를 Java로 전송
- 이후 Java에서 input/expect 존재 여부를 기준으로 필터링 수행

API 성능 최적화 리팩토링

2. 조회 데이터 양 최적화 **No 2. 해당 디자인의 File과 Function** **그리고 TC 목록을 반환하는 API**

```
tc_agg AS (  
  SELECT functionid, tcid,  
         MAX(CASE WHEN valuetype = 'description' THEN value END) AS description  
  FROM designfunctiontestcase  
 WHERE designid = (SELECT id FROM di)  
    AND valuetype IN ('input', 'expect', 'description')  
  GROUP BY functionid, tcid  
  HAVING SUM(CASE WHEN valuetype IN ('input', 'expect') THEN 1 ELSE 0 END) > 0  
  -- ↑ input/expect 없는 TC(description만 있는 TC)를 DB 단에서 제거  
)
```

DB 단에서 필터링하게 쿼리 개선

```
79 -  
80 -      TcAssembly tc = tcMap.get(functionId).get(tcId);  
81 -      if ("input".equals(valuetype) || "expect".equals(valuetype)) {  
82 -          tc.hasInputOrExpect = true; // 이 TC에 input or expect 하나라도 있으면 유효체크  
83 -      } else if ("description".equals(valuetype)) { // description 이면 따로 저장  
84 -          tc.description = value;  
85 -      }  
86 -  }
```

서비스단 필터링 로직 제거

AFTER

- DB에서 GROUP BY + HAVING을 통해 유효한 TC만 사전 필터링
- TC 단위로 집계하여 필요한 데이터만 반환

RESULT

- 전송 데이터 약 101,000 → 12,000 수준으로 감소
- 서비스 단 필터링 로직 제거
- 네트워크, 메모리, 가공 비용 모두 감소

API 성능 최적화 리팩토링

2-2. 복합키 기반 조회로 데이터 정합성 개선

No 3. 필요한 TC를 선택하고,
해당 TC의 실제 정보를 반환하는 API

```
// 선택된 (filePath, signature) 조합에 해당하는 함수 정보 일괄 조회 (getDetailTcData용)  
// 반환: [0]=filePath, [1]=functionId, [2]=signature, [3]=spec  
@Query("SELECT s.path, f.id, f.signature, f.spec " +  
    "FROM designsourceinfo s, designfunctioninfo f " +  
    "WHERE s.designId = :designId AND f.designId = :designId " +  
    "AND f.sourceId = s.id " +  
    "AND s.path IN :filePaths AND f.signature IN :signatures")  
List<Object[]> findFunctionsByDesignIdAndPathsAndSignatures(  
    @Param("designId") String designId,
```

```
@Query("SELECT d.functionId, d.tcId, d.valueType, d.valueName, d.value " +  
    "FROM designfunctiontestcase d " +  
    "WHERE d.functionId IN :functionIds AND d.tcId IN :tcIds " +  
    "AND d.valueType IN ('input', 'expect', 'description')")  
List<Object[]> findTcValuesByFunctionIdsAndTcIds(  
    @Param("functionIds") List<String> functionIds,  
    @Param("tcIds") List<String> tcIds);
```

기존에는 복합키(쌍)로 조회한 게 아니라,
각 컬럼을 따로따로 IN 조건으로 조회

BEFORE

- filePath IN + signature IN / functionId IN + tcId IN 방식으로 조회
- 서로 독립적인 IN 조건으로 인해
- 요청하지 않은 조합까지 포함되는 크로스 매칭 발생
- 동일 fuctionId가 여러 함수에 존재할 경우 함수 간 데이터 오염 발생

API 성능 최적화 리팩토링

2-2. 복합키 기반 조회로 데이터 정합성 개선

No 3. 필요한 TC를 선택하고,
해당 TC의 실제 정보를 반환하는 API

```
FROM designsourceinfo s ON s.designid = di.id " +  
"JOIN designfunctioninfo f ON f.sourceid = s.id AND f.designid = di.id " +  
"WHERE di.name = :designName " +  
"AND (s.path || CHR(1) || f.signature) IN :pairs")  
List<CtDetailTcDataResponse.FunctionInfoRow> findFunctionsByDesignNameAndPairs(  
    @Param("designName") String designName,  
    @Param("pairs") List<String> pairs);
```

```
"    d.value AS value " +  
"FROM designfunctiontestcase d " +  
"WHERE (d.functionid || CHR(1) || d.tcid) IN :pairs " +  
"AND d.valuetype IN ('input', 'expect', 'description') " +  
"ORDER BY d.functionid ASC, d.tcid ASC")  
List<CtDetailTcDataResponse.TcValueRow> findTcValuesByPairs(  
    @Param("pairs") List<String> pairs);
```

복합 식별 조건 기반 조회로 잘못된 조합을 원천 차단하고,
데이터 정합성과 조회 정확도를 함께 개선

AFTER

- (filePath, signature), (functionId, tcId) 복합 조건 기반 조회로 변경
- 요청한 쌍만 정확히 매칭하도록 쿼리 재설계

RESULT

- 크로스 프로젝트 및 데이터 오염 문제 제거
- 불필요한 row 조회 감소
- 서비스 단 중복 제거 및 보정 로직 제거
- 데이터 정합성 및 조회 정확도 향상

API 성능 최적화 리팩토링

3. 서비스 단 가공 최적화 No 2. 해당 디자인의 File과 Function 그리고 TC 목록을 반환하는 API

```
// 최종 응답 조립
// CtFileFunctionTcListsResponse.FileEntry: filePath + FunctionEntry 리스트
List<CtFileFunctionTcListsResponse.FileEntry> files = new ArrayList<>();
for (Map.Entry<String, List<Object[]>> sourceEntry : sourceMap.entrySet()) {
    String filePath = sourceEntry.getKey();
    // CtFileFunctionTcListsResponse.FunctionEntry: functionSignature + TcEntry 리스트
    List<CtFileFunctionTcListsResponse.FunctionEntry> functionEntries = new ArrayList<>();

    for (Object[] funcRow : sourceEntry.getValue()) {
        String functionId = (String) funcRow[1];
        String signature = (String) funcRow[2];

        Map<String, TcAssembly> tcsByFunction =
            tcMap.getOrDefault(functionId, Collections.emptyMap());

        // input/expect가 하나라도 있는 TC만 포함
        // CtFileFunctionTcListsResponse.TcEntry: testCaseCode(TC ID) + description
        List<CtFileFunctionTcListsResponse.TcEntry> tcEntries = tcsByFunction.entrySet().stream()
            .filter(e -> e.getValue().hasInputOrExpect())
            .map(e -> CtFileFunctionTcListsResponse.TcEntry.builder()
                .testCaseCode(e.getKey())
                .description(e.getValue().description)
                .build())
            .collect(Collectors.toList());

        if (tcEntries.isEmpty()) continue;

        functionEntries.add(CtFileFunctionTcListsResponse.FunctionEntry.builder()
            .functionSignature(signature)
            .testCases(tcEntries) // TcEntry 리스트
            .build());
    }
}
```

Map 두 개를 메모리에 올리고, 이 둘을 다시
교차 참조하며 결과를 조립하는 2중 for문 구조

```
// TC 데이터 조립: Map<functionId, Map<tcId, TcAssembly>>
Map<String, Map<String, TcAssembly>> tcMap = new LinkedHashMap<>();
for (Object[] row : tcRows) {
    String functionId = (String) row[0];
    String tcId = (String) row[1];
    String valueType = (String) row[2];
    String value = (String) row[3];

    tcMap.computeIfAbsent(functionId, k -> new LinkedHashMap<>())
        .computeIfAbsent(tcId, k -> new TcAssembly());

    TcAssembly tc = tcMap.get(functionId).get(tcId);
    if ("input".equals(valueType) || "expect".equals(valueType)) {
        tc.hasInputOrExpect = true; // 이 TC에 input or expect 하나라도 있으면 유효치크
    } else if ("description".equals(valueType)) { // description 이면 따로 저장
        tc.description = value;
    }
}
```

BEFORE

- TC rows를 functionId → tcId → TcAssembly 형태의 Map으로 조립
- source/function rows를 filePath → function rows 형태의 Map으로 별도 조립
- 이후 sourceMap을 돌면서 내부에서 다시 tcMap을 찾아 붙이는 이중 참조 구조
- 즉, 중간 자료구조를 여러 개 만든 뒤 다시 찾아가며 붙이는 방식

```
// 소스파일+함수 조립: Map<filePath, List<Object[]>> (순서 유지)
Map<String, List<Object[]>> sourceMap = new LinkedHashMap<>();
for (Object[] row : sourceFunctionRows) {
    String path = (String) row[0];
    sourceMap.computeIfAbsent(path, k -> new ArrayList<>()).add(row);
}
```

API 성능 최적화 리팩토링

3. 서비스 단 가공 최적화 **No 2. 해당 디자인의 File과 Function 그리고 TC 목록을 반환하는 API**

```
for (CtFileFunctionTcListsResponse.FlatRow row : flatRows) {
    // 파일 변경 감지: 이전 function/file을 환경(flush) 후 새 file 시작
    if (!row.getFilePath().equals(currentFilePath)) {
        flushFunction(currentFunctionId, currentFunctionSig, currentTcs, currentFunctions);
        flushFile(currentFilePath, currentFunctions, files);
        currentFilePath = row.getFilePath();
        currentFunctions = new ArrayList<>();
        currentFunctionId = null;
        currentTcs = null;
    }
    // 함수 변경 감지: 이전 function을 환경(flush) 후 새 function 시작
    if (!row.getFunctionId().equals(currentFunctionId)) {
        flushFunction(currentFunctionId, currentFunctionSig, currentTcs, currentFunctions);
        currentFunctionId = row.getFunctionId();
        currentFunctionSig = row.getFunctionSignature();
        currentTcs = new ArrayList<>();
    }
    // 현재 함수의 TC 누락 (TC 없는 함수의 null 행은 skip)
    if (row.getTestCaseCode() != null) {
        currentTcs.add(CtFileFunctionTcListsResponse.TcEntry.builder()
            .testCaseCode(row.getTestCaseCode())
            .description(row.getDescription() != null ? row.getDescription() : "")
            .build());
    }
}
```

이중 Map 기반 교차 참조 구조를 제거하고,
정렬된 flat row 기반 Pass Pattern 조립 방식 교체

AFTER

- 1번에서 개선된 쿼리 결과는 Flat하고 정렬해서 일괄로 데이터를 올려줌
- 이를 이용해 앞에서부터 한 번만 순회하는 Pass Pattern으로 nested 구조를 만들도록 변경했다.

RESULT

- tcMap, sourceMap 두 개의 중간 Map 제거
- TcAssembly 내부 클래스 제거
- 이중 반복문 → 단일 순회 구조로 단순화
- Map 엔트리와 임시 객체 생성이 줄어들어 서비스 단 메모리 사용량 감소

API 성능 최적화 리팩토링

3. 서비스 단 가공 최적화

쿼리 결과물 - Order된 Flat한 일괄 데이터 묶음



a.c	F1	void initMotor()	TC-001	power on
a.c	F1	void initMotor()	TC-002	sensor check
a.c	F1	void initMotor()	TC-003	init complete
a.c	F2	int readSpeed()	TC-004	normal speed
a.c	F2	int readSpeed()	TC-005	max speed
a.c	F2	int readSpeed()	TC-006	sensor error
b.c	F3	void stopMotor()	TC-007	emergency stop
b.c	F3	void stopMotor()	TC-008	timeout stop
b.c	F4	void resetMotor()	TC-009	reset signal
b.c	F4	void resetMotor()	TC-010	reset complete
c.c	F5	void checkTemp()	TC-011	normal temp
c.c	F5	void checkTemp()	TC-012	overheat warning
c.c	F6	void coolDown()	TC-013	fan start
c.c	F6	void coolDown()	TC-014	temp decrease

처음부터 끝까지 딱 한번만 순회하며 경계가 바뀔때를 감지

Pass 패턴 ?

경계값 기준으로 순차적으로 계층 분리

1. filePath 바뀌면 → 이전 file 확정
2. functionId 바뀌면 → 이전 function 확정
3. 같으면 → TC 계속 누적
4. 마지막 → 남은 것 flush

즉, "찾아가며 붙이는 구조"에서 "흐르는 순서대로 조립하는 구조"로 바꾼 것

API 성능 최적화 리팩토링

3. 서비스 단 가공 최적화

Map vs Pass 패턴

MAP

- 흩어져 있는 데이터를 키로 빠르게 찾아야 할 때 유리
- 순서가 보장되지 않아도 접근 가능
- 대신 중간 자료구조와 메모리 사용이 증가할 수 있음
- 또한 Map 객체를 만드는 리소스도 고려해야함

Pass 패턴

- 데이터가 정렬되어 연속적으로 모여 있을 때 유리
- 앞에서부터 한 번만 훑으면서 조립 가능
- 대신 정렬과 그룹 경계가 보장되어야 함

⇒ 이 특성에 착안해서 No3. API 성능 개선에 적용

API 성능 최적화 리팩토링

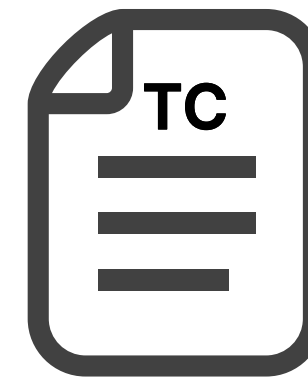
3. 서비스 단 가공 최적화

No 3. 필요한 TC를 선택하고,
해당 TC의 실제 정보를 반환하는 API

⇒ 사용자 시나리오를 생각해 보았을때...



사용자가 보통 띄엄띄엄 선택한다고 판단



해당 함수의 테스트 케이스는
연속적으로 쭉쭉 선택하는 경우가 많다고 판단

API 성능 최적화 리팩토링

3. 서비스 단 가공 최적화

No 3. 필요한 TC를 선택하고,
해당 TC의 실제 정보를 반환하는 API

```
for (Map.Entry<String, List<String>> entry : funcIdCodes.entrySet()) {
    CtDetailTcDataResponse.FunctionInfoRow info = funcKeyInfo.get(entry.getKey());
    if (info == null) {
        // 요청한 (filePath, signature) 지능기 대응 없음
        notFound.add(entry.getKey().replace("\\u0001", "\\u0000"));
        continue;
    }

    Map<String, CtDetailTcDataResponse.TcEntry> tcByCode = tcListByFuncId
        .getOrDefault(info.getFunctionId(), Collections.emptyMap());

    // 요청한 TC 중 대응 있는 것 찾기
    for (String tcId : entry.getValue()) {
        if (!tcByCode.containsKey(tcId)) {
            notFound.add(info.getFilePath());
        }
    }

    // Pass Pattern => TC 검색 변경 길이
    for (CtDetailTcDataResponse.TcValueRow r : tcRows) {
        if (r.getFunctionId().equals(prevFuncId) || r.getTcId().equals(prevTcId)) {
            flushTcEntry(prevFuncId, prevTcId, curDesc, curValues, tcListByFuncId);
            prevFuncId = r.getFunctionId();
            prevTcId = r.getTcId();
            curDesc = "";
            curValues = new ArrayList<>();
        }
        if ("description".equals(r.getValueType())) {
            curDesc = r.getValue() != null ? r.getValue() : "";
        } else {
            curValues.add(CtDetailTcDataResponse.TcValue.builder()
                .valueName(r.getValueName())
                .valueType("input".equals(r.getValueType()) ? "Input" : "Expected")
                .value(r.getValue())
                .build());
        }
    }
    flushTcEntry(prevFuncId, prevTcId, curDesc, curValues, tcListByFuncId); // 마지막 TC 포함
}
```

상위 선택 구조는 Map,
하위 TC 조립은 Pass Pattern으로 역할 분리

AFTER

- file / function: 사용자가 띄엄띄엄 선택할 수 있으므로 Map 기반 접근
- TC 상세 조립: 동일 function 내에서 연속적으로 조회되는 흐름이 많다고 보고 Pass Pattern 적용
- ORDER BY functionId, tcId 기준으로 TC를 순차 조립

RESULT

- TC 조립 구조 단순화 → 중간 Map 및 내부 클래스 감소
- 중복 제거 / 보정 로직 축소
- 사용자 선택 흐름에 맞는 가공 구조로 최적화

API 성능 최적화 리팩토링

결론 정리

개선된 점

- 1) 정확성 (버그 수정) → 동일 함수가 여러 개일 때 발생하던 버그 제거
- 2) Round Trip 최적화 → DB 부하 감소
- 3) 서비스 단 가공 비용 감소 (JVM 메모리 부담 감소)

Trade Off

- DB 처리 부하 증가 → 내부적으로 더 무거운 연산을 수행
- 문자열 결합 기반 IN 조건(복합키) → 인덱스 활용 불가

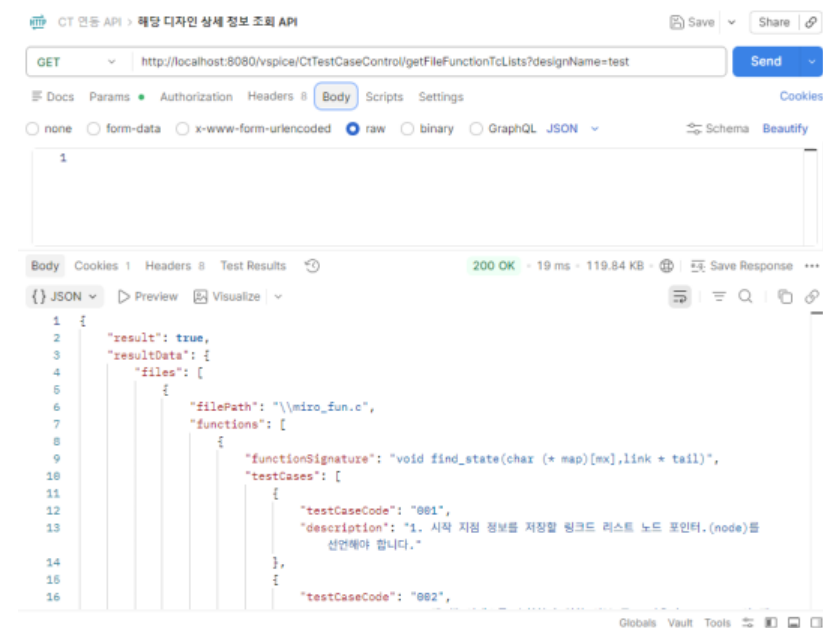
03 CT-API 결과

SURESOFT

API 성능 측정

일반 상황 성능 측정 방식 테스트용 Miro 프로젝트 기준 (TC 233개 기준)

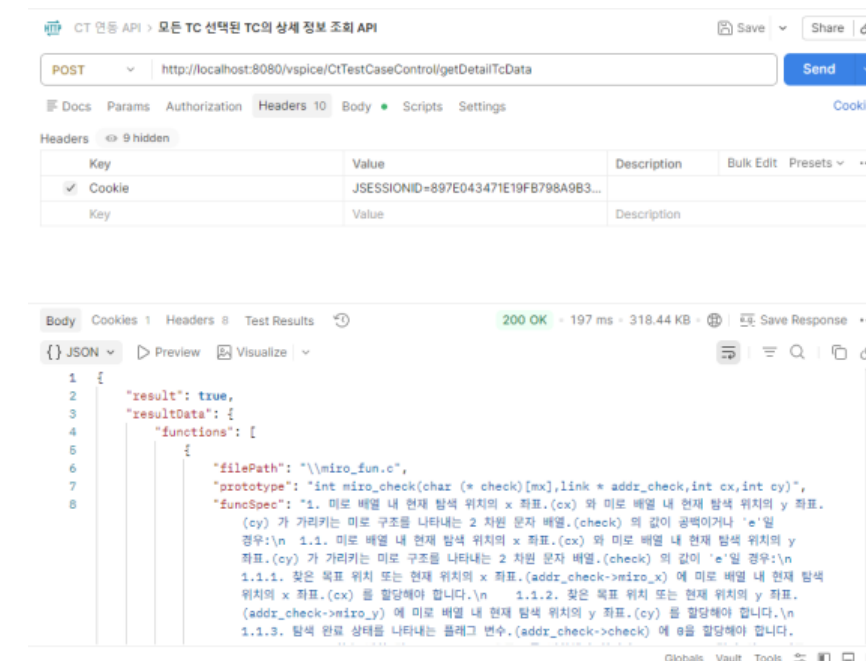
모든 목록 조회 - 1040 rows



No 2. 해당 디자인의 File과 Function
그리고 TC 목록을 반환하는 API

19ms - 119.84KB

모든 TC 조회 - 13000 rows



No 3. 필요한 TC를 선택하고,
해당 TC의 실제 정보를 반환하는 API

197ms - 318.44KB

03 CT-API 결과

SURESOFT

API 성능 측정

극단 상황 성능 측정 방식

테스트 데이터 구성

AI 기반으로 테스트용 디자인 시드를 생성하여 대용량 데이터셋 구성

테스트 데이터 기준

(약 10만 TC)

file: 270개

function: 13,500개 (270 × 50)

TC: 101,250개

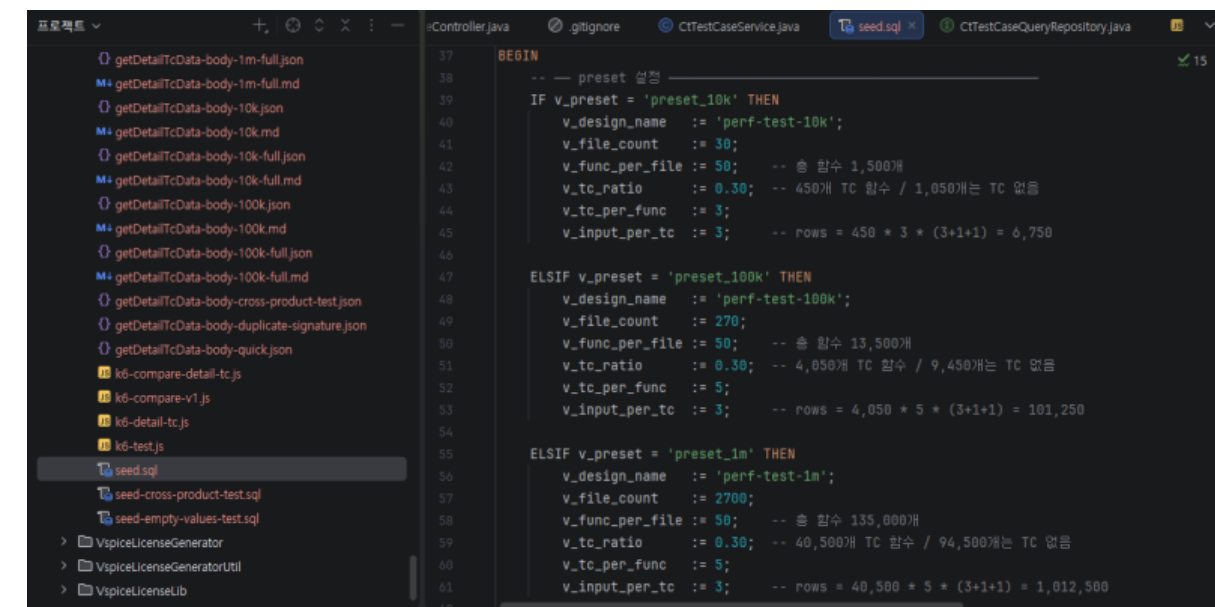
(약 100만 TC)

file: 2,700개

function: 135,000개 (2,700 × 50)

TC: 1,012,500개

청크 단위 분할 필요 ⇒ 폐기



```
37 BEGIN
38 -- preset 설정
39 IF v_preset = 'preset_10k' THEN
40   v_design_name := 'perf-test-10k';
41   v_file_count  := 30;
42   v_func_per_file := 50; -- 총 함수 1,500개
43   v_tc_ratio     := 0.30; -- 450개 TC 함수 / 1,050개는 TC 없음
44   v_tc_per_func  := 3;
45   v_input_per_tc := 3; -- rows = 450 * 3 * (3+1+1) = 6,750
46
47 ELSIF v_preset = 'preset_100k' THEN
48   v_design_name := 'perf-test-100k';
49   v_file_count  := 270;
50   v_func_per_file := 50; -- 총 함수 13,500개
51   v_tc_ratio     := 0.30; -- 4,050개 TC 함수 / 9,450개는 TC 없음
52   v_tc_per_func  := 5;
53   v_input_per_tc := 3; -- rows = 4,050 * 5 * (3+1+1) = 101,250
54
55 ELSIF v_preset = 'preset_1m' THEN
56   v_design_name := 'perf-test-1m';
57   v_file_count  := 2700;
58   v_func_per_file := 50; -- 총 함수 135,000개
59   v_tc_ratio     := 0.30; -- 40,500개 TC 함수 / 94,500개는 TC 없음
60   v_tc_per_func  := 5;
61   v_input_per_tc := 3; -- rows = 40,500 * 5 * (3+1+1) = 1,012,500
```

03 CT-API 결과

SURESOFT

API 성능 측정

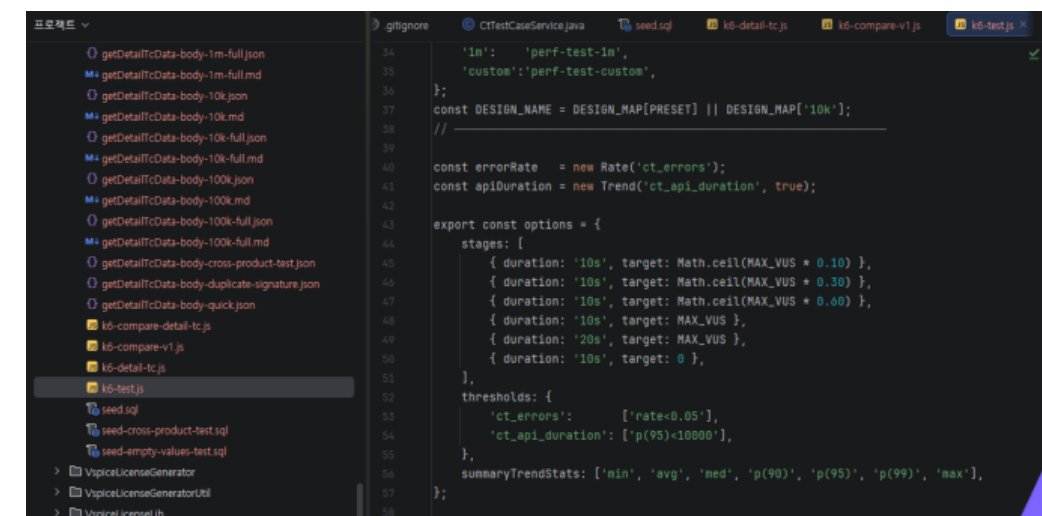
극단 상황 성능 측정 방식

측정 도구

- K6를 사용해 반복 호출 및 응답 시간 측정
- 본래 부하 테스트 도구이지만, 이번에는 반복 실행을 통한 평균 응답 속도와 안정성 확인 용도로 활용

측정 방식

- 동일 조건에서 30회 반복 실행
- 각 실행 결과를 바탕으로 평균 응답 속도 산출
- 반복 측정을 통해 안정적으로 동작하는지 함께 확인



```
1  'in': 'perf-test-in',
2  'custom': 'perf-test-custom',
3  };
4  const DESIGN_NAME = DESIGN_MAP[PRESET] || DESIGN_MAP['10k'];
5  //
6  const errorRate = new Rate('ct_errors');
7  const apiDuration = new Trend('ct_api_duration', true);
8  //
9  export const options = {
10   stages: [
11     { duration: '10s', target: Math.ceil(MAX_VUS * 0.10) },
12     { duration: '10s', target: Math.ceil(MAX_VUS * 0.30) },
13     { duration: '10s', target: Math.ceil(MAX_VUS * 0.60) },
14     { duration: '10s', target: MAX_VUS },
15     { duration: '20s', target: MAX_VUS },
16     { duration: '10s', target: 0 },
17   ],
18   thresholds: {
19     'ct_errors': ['rate<0.05'],
20     'ct_api_duration': ['p(95)<10000'],
21   },
22   summaryTrendStats: ['min', 'avg', 'med', 'p(90)', 'p(95)', 'p(99)', 'max'],
23 };
24
```

테스트 조건 별로 프리셋을 만들어 테스트 자동화

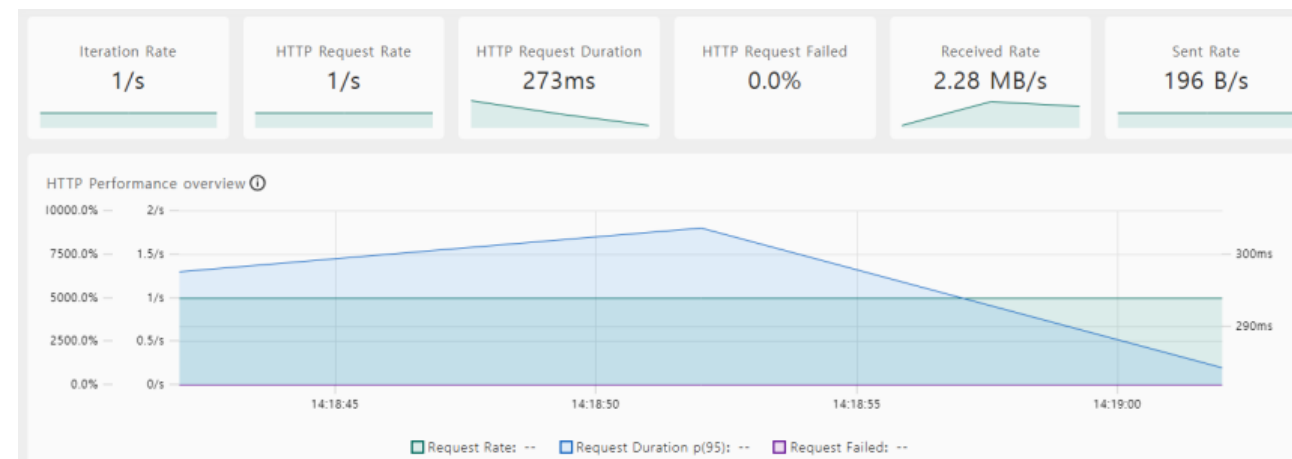
k6

02 CT-API 구현 및 개선

SURESOFT

API 구현

리팩토링 전 - 성능 측정 **BEFORE**



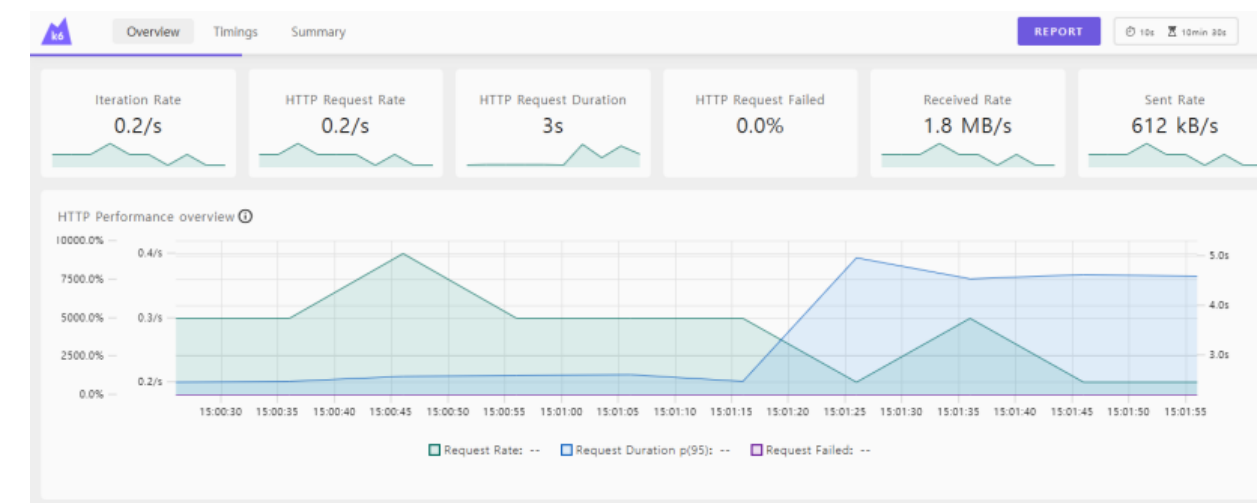
No 2. 해당 디자인의 File과 Function
그리고 TC 목록을 반환하는 API 약 273ms

(약 10만 TC)

file: 270개

function: 13,500개 (270 × 50)

TC: 101,250개



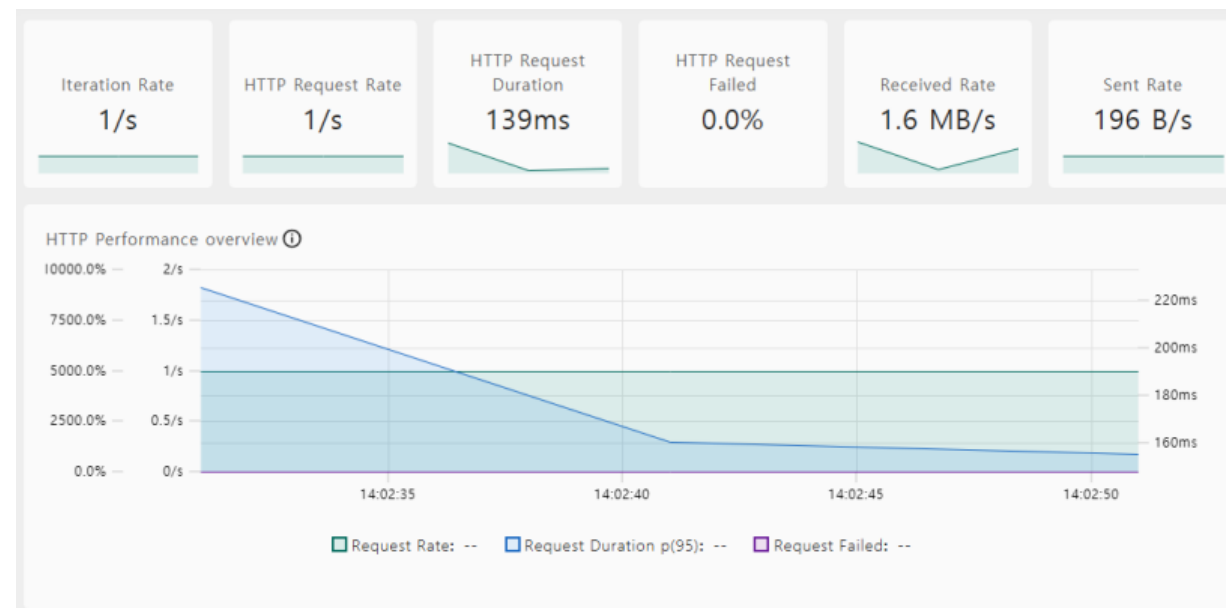
No 3. 필요한 TC를 선택하고,
해당 TC의 실제 정보를 반환하는 API 약 3s

03 CT-API 결과

SURESOFT

API 성능 측정

리팩토링 후 - 성능 측정 **AFTER**



No 2. 해당 디자인의 File과 Function
그리고 TC 목록을 반환하는 API

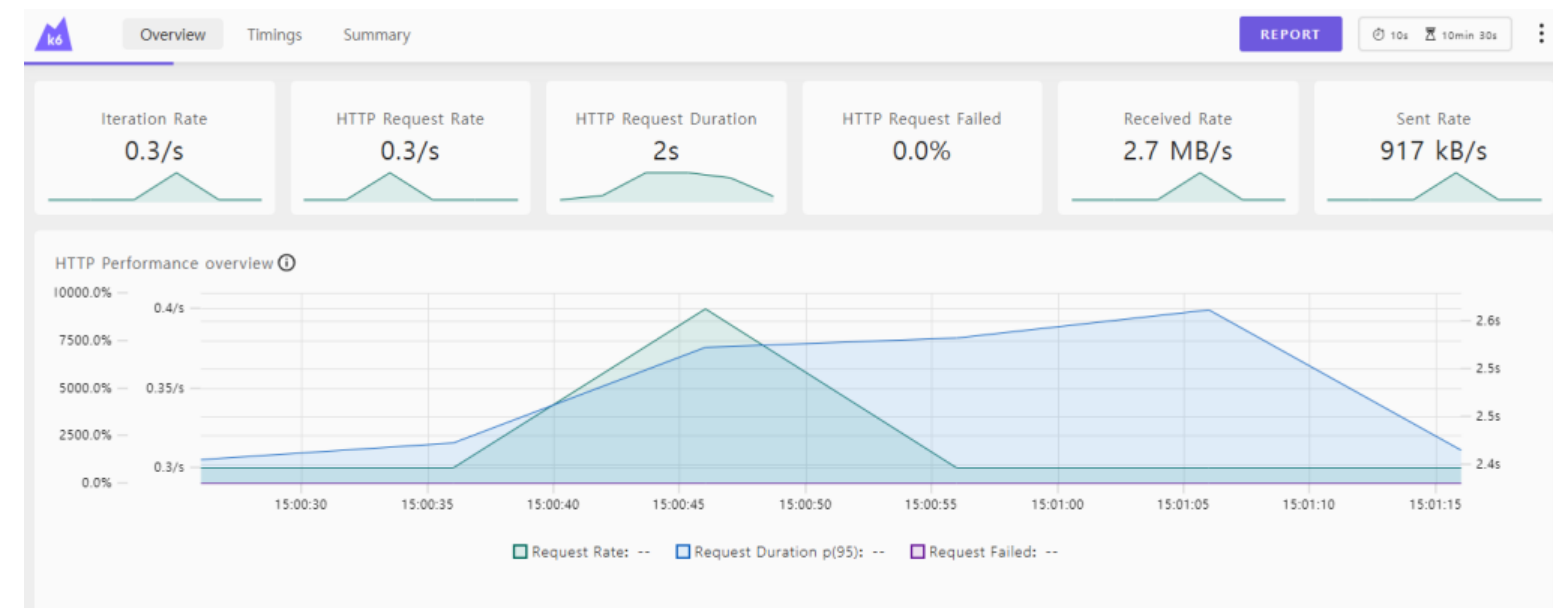
약 139ms

(약 10만 TC)

file: 270개

function: 13,500개 (270 × 50)

TC: 101,250개



No 3. 필요한 TC를 선택하고,
해당 TC의 실제 정보를 반환하는 API

약 2s

API 성능 측정

리팩토링 전 후 - 성능 비교

No.2 API (목록 조회 API)

약 10만 Rows 기준 - json 1.5MB

- 리팩토링 전: 약 273ms
 - 리팩토링 후: 약 139ms
- ⇒ 약 49% 개선 (거의 절반 수준으로 감소)

No.3 API (상세 조회 API)

약 64만 Rows 기준 - json 8.5MB

- 리팩토링 전: 약 3,000ms (3s)
 - 리팩토링 후: 약 2000ms
- ⇒ 약 33% 개선

체감 성능이 크게 개선된 것을 확인 가능 !

API 성능 측정

CT측 추가 요청사항

No.2 API (목록 조회 API)

TC 없는 함수도 포함해서 내려달라

- TC 없는 함수는 빈 배열로라도 전달
⇒ TC 유무와 상관 없이 모든 함수 전달

No.3 API (상세 조회 API)

TC 상세 API에 caller / callee 정보 추가

- function info에
 - caller function 리스트
 - callee function 리스트
 - 같이 내려달라

API 성능 측정

CT측 추가 요청사항

No.2 API (목록 조회 API)

TC 없는 함수도 포함해서 내려달라

- TC 없는 함수는 빈 배열로라도 전달
⇒ TC 유무와 상관 없이 모든 함수 전달

No.3 API (상세 조회 API)

TC 상세 API에 caller / callee 정보 추가

- function info에
 - caller function 리스트
 - callee function 리스트
 - 같이 내려달라



```
@@ -37,8 +35,8 @@ public interface CtTestCaseQueryRepository extends JpaRepository<DesignFunctionT
    "FROM di " +
    "JOIN designsourceinfo s ON s.designid = di.id AND s.isheader = false " +
    "JOIN designfunctioninfo f ON f.sourceid = s.id AND f.designid = di.id AND f.active = true " +
-    "JOIN tc_agg ta ON ta.functionid = f.id " +
-    "ORDER BY s.path ASC, f.name ASC, f.id ASC, ta.tcid ASC")
+    "LEFT JOIN tc_agg ta ON ta.functionid = f.id " +
+    "ORDER BY s.path ASC, f.name ASC, f.id ASC, ta.tcid ASC NULLS FIRST")
    List<CtFileFunctionTcListsResponse.FlatRow> findFlatByDesignName(@Param("designName") String designName);
```

- LEFT JOIN 적용 ⇒ TC 없는 함수도 row 생성
- 서비스 단 tcs.isEmpty 필터 제거

```
@@ -54,7 +52,11 @@ public interface CtTestCaseQueryRepository extends JpaRepository<DesignFunctionT
    "SELECT s.path AS filePath, " +
    " f.id AS functionId, " +
    " f.signature AS signature, " +
-    " f.spec AS spec " +
+    " f.spec AS spec, " +
+    "(SELECT string_agg(c.name, CHR(1)) FROM designcallfunctioninfo c " +
+    " WHERE c.functionid = f.id AND c.type IN ('Called', 'Caller')) AS callerNames, " +
+    "(SELECT string_agg(c.name, CHR(1)) FROM designcallfunctioninfo c " +
+    " WHERE c.functionid = f.id AND c.type IN ('Callee', 'Calling')) AS calleeNames " +
```

- callerNames / calleeNames를 같이 붙여서 가져오는 서브쿼리 추가
- (Called, Caller) / (Callee, Calling) 모두 대응

추가 요구사항 구현 및
테스트 후 전달 완료

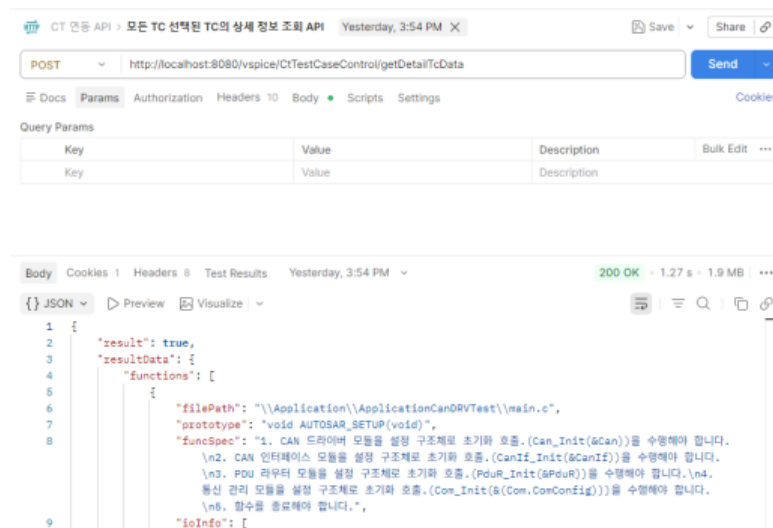
03 CT-API 결과

SURESOFT

API 성능 측정

추가 리팩토링 진행

No.3 API (상세 조회 API)



문제 발견

- 실제 ECU 코드 기반 중간 규모 프로젝트에서도 응답 속도 과도하게 느림
- 771개 TC → 약 1.2~1.5초
- 반면 100k TC 조회도 약 2초 수준 → 비정상적인 성능 패턴
→ 단순 데이터 양 문제가 아닌 구조적 병목 의심

```
06-Apr-2026 15:59:08.725 정보 [main] or
다.
P[PERF] Q1 funcRows=57 → 41ms
[PERF] Q2 ioRows=876 → 17ms
[PERF] Q3 tcRows=9425 → 1447ms
k[PERF] Java조립 → 87ms
[PERF] 전체 → 1592ms
```

중간중간 로그를 넣어 단계별 실행 시간을 측정

→ 전체 지연의 대부분이 Q3 쿼리에서 발생하는 것을 확인

03 CT-API 결과

SURESOFT

API 성능 측정

추가 리팩토링 진행

No.3 API (상세 조회 API)

- Query 3 분석

메인 원인

(functionid || tcid) IN (...) 구조

⇒ 컬럼에 연산(||)이 들어가면 인덱스 못탐

인덱스 무효화 → Full Scan 발생

테이블 풀스캔 → 행마다 문자열 연산 → pairs IN 비교

해결

- 컬럼 연산 제거 → functionid 인덱스 직접 활용
- DB에서는 인덱스를 활용할 수 있는 조건으로 넓게 조회하고, 정확한 (functionid, tcid) 매칭은 서비스 단 Set 후처리로 분리

```
// ORDER BY functionid, tcid → 서비스 pass 패턴 호환
//
// alias → TcValueRow getter: functionId/tcId/valueType/valueName/v
@Query(nativeQuery = true, value =
    "SELECT d.functionid AS functionId, " +
    "      d.tcid AS tcId, " +
    "      d.valuetype AS valueType, " +
    "      d.valuename AS valueName, " +
    "      d.value AS value " +
    "FROM designfunctiontestcase d " +
    "WHERE (d.functionid || CHR(1) || d.tcid) IN :pairs " +
    "AND d.valuetype IN ('input', 'expect', 'description') " +
    "ORDER BY d.functionid ASC, d.tcid ASC")
List<CtDetailTcDataResponse.TcValueRow> findTcValuesByPairs(
    @Param("pairs") List<String> pairs);

// alias → TcValueRow getter: functionId/tcId/valueType/valueName/v
@Query(nativeQuery = true, value =
    "SELECT d.functionid AS functionId, " +
    "      d.tcid AS tcId, " +
    "      d.valuetype AS valueType, " +
    "      d.valuename AS valueName, " +
    "      d.value AS value " +
    "FROM designfunctiontestcase d " +
    "WHERE d.functionid IN :functionIds " +
    "AND d.valuetype IN ('input', 'expect', 'description') " +
    "ORDER BY d.functionid ASC, d.tcid ASC " +
    "      CASE lower(d.valuetype) WHEN 'input' THEN 1 WHEN 'expect' THEN 2 WHEN 'description' THEN 3 " +
    "      d.valuename ASC NULLS LAST")
List<CtDetailTcDataResponse.TcValueRow> findTcValuesByFunctionIds(
    @Param("functionIds") List<String> functionIds);
```

```
// Q3: functionId IN 기반 조회 → functionid 인덱스 활용
// 요청된 (functionid, tcid) 쌍 Set으로 후처리 필터링
Set<String> requestedPairs = new HashSet<>();
for (Map.Entry<String, List<String>> entry : funcToTcCodes.entrySet()) {
    CtDetailTcDataResponse.FunctionInfoRow info = funcKeyToInfo.get(entry.getKey());
    if (info == null) continue;
    for (String tcId : entry.getValue()) {
        requestedPairs.add(info.getFunctionId() + "\u0001" + tcId);
    }
}
```

1. 요청된 pair → Set으로 미리 저장
2. DB → functionId 기준으로 빠르게 조회 (인덱스 사용)
3. Java → Set.contains()로 정확히 필터링

```
// Pass Pattern => TC 검색 변경 감지
// requestedPairs로 요청하지 않은 TC 행 건너뛰기 (functionid IN 확장 범위 필터링)
for (CtDetailTcDataResponse.TcValueRow r : tcRows) {
    if (!requestedPairs.contains(r.getFunctionId() + "\u0001" + r.getTcId())) continue;
    if (!r.getFunctionId().equals(prevFuncId) || !r.getTcId().equals(prevTcId)) {
        flushTcEntry(prevFuncId, prevTcId, curDesc, curValues, tcListByFuncId);
    }
}
```

⇒ Q3 병목 해소

API 성능 측정

추가 리팩토링 진행

No.3 API (상세 조회 API) - Query 1 분석

```
//
// alias + FunctionInfoRow getter: filePath/functionId/signature/spec
@Query(nativeQuery = true, value =
    "SELECT s.path AS filePath, " +
    " f.id AS functionId, " +
    " f.signature AS signature, " +
    " f.spec AS spec, " +
    " (SELECT string_agg(c.name, CHR(1)) FROM designcallfunctioninfo c " +
    " WHERE c.functionid = f.id AND c.type IN ('Called', 'Caller')) AS callerNames, " +
    " (SELECT string_agg(c.name, CHR(1)) FROM designcallfunctioninfo c " +
    " WHERE c.functionid = f.id AND c.type IN ('Callee', 'Calling')) AS calleeNames " +
    "FROM designinfo di " +
    "JOIN designsourceinfo s ON s.designid = di.id " +
    "JOIN designfunctioninfo f ON f.sourceid = s.id AND f.designid = di.id " +
    "WHERE di.name = :designName " +
    "AND (s.path || CHR(1) || f.signature) IN :pairs")
List<CtDetailToDataResponse.FunctionInfoRow> findFunctionsByDesignNameAndPairs(
    @Param("designName") String designName,
    @Param("pairs") List<String> pairs);

// caller/callee: 코릴레이터드 서브쿼리(N*2번) + LEFT JOIN 집계(1번)로 교체
// alias + FunctionInfoRow getter: filePath/functionId/signature/spec/callerNames/calleeNames
@Query(nativeQuery = true, value =
    "SELECT s.path AS filePath, " +
    " f.id AS functionId, " +
    " f.signature AS signature, " +
    " f.spec AS spec, " +
    " cfi.callerNames AS callerNames, " +
    " cfi.calleeNames AS calleeNames " +
    "FROM designinfo di " +
    "JOIN designsourceinfo s ON s.designid = di.id " +
    "JOIN designfunctioninfo f ON f.sourceid = s.id AND f.designid = di.id " +
    "LEFT JOIN ( " +
    " SELECT functionid, " +
    " string_agg(name, CHR(1) ORDER BY name) FILTER (WHERE type IN ('Called', 'Caller')) AS callerNames, " +
    " string_agg(name, CHR(1) ORDER BY name) FILTER (WHERE type IN ('Callee', 'Calling')) AS calleeNames " +
    " FROM designcallfunctioninfo " +
    " GROUP BY functionid " +
    " ) cfi ON cfi.functionid = f.id " +
    "WHERE di.name = :designName "
```

보조 원인

반환되는 함수 N개마다 서브쿼리가 2번씩 실행
함수 57개 → 내부적으로 114번 서브쿼리 실행

해결

- 서브쿼리 → LEFT JOIN 집계로 교체
- 한번에 일괄 조회 후 JOIN하는 방식으로 쿼리 폭증 방지

⇒ Q1 N+1 문제 해결

03 CT-API 결과

SURESOFT

API 성능 측정

추가 개선점

Gzip 이나 Brotli와 같은 json 압축 기법 적용

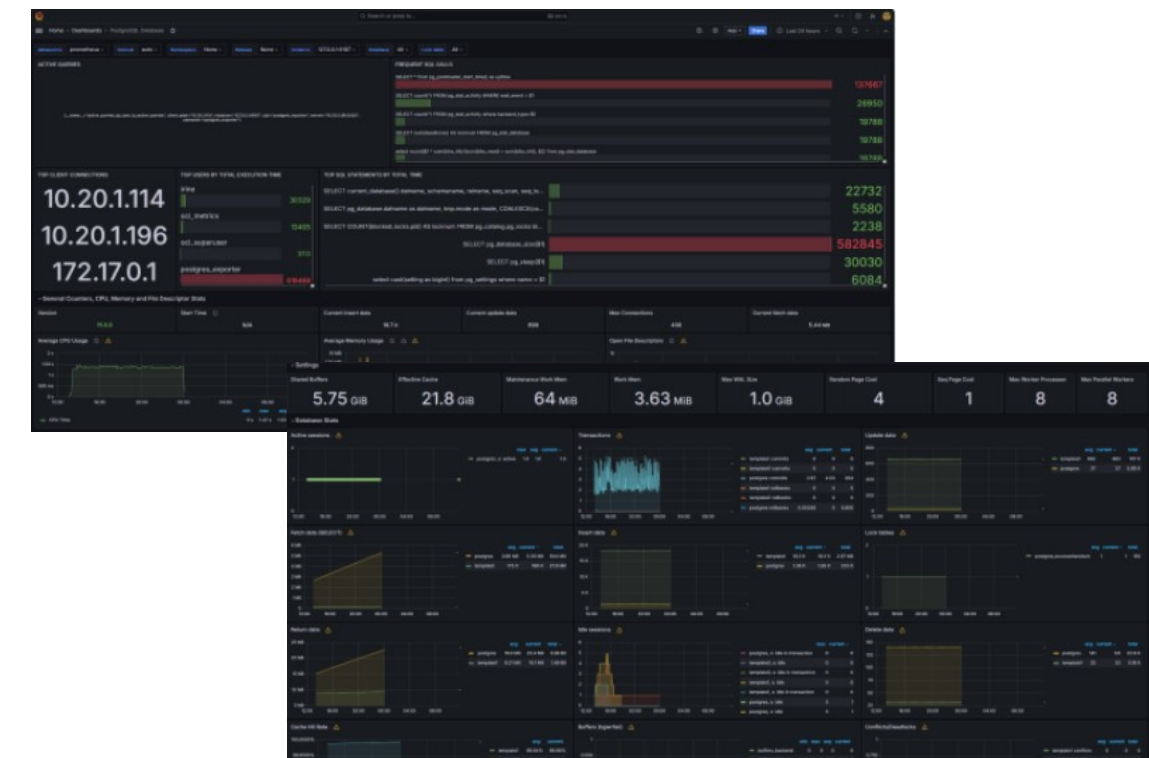
⇒ CT 측 환경이 애매해서 추후 필요시 도입



Grafana 기반 DB 부하 모니터링

⇒ 프로메테우스 & PgExporter 기반으로 모니터링 구축은 완료

⇒ 하지만 사용법이 어려워 이번에는 분석에 쓰지 못했음



감사합니다!